

Subject Name: OBJECT ORIENTED PROGRAMMING DESIGN

Subject Code: CS T33

2 MARKS AND 11 MARKS FOR ALL UNITS

UNIT II

Constructors and Destructors: Purpose of Constructors and Destructors – overloading constructors – constructors with default arguments – copy constructors – calling constructors and destructors – dynamic initialization using constructors – recursive constructor.

Overloading Functions: Overloading unary operators – constraint on increment and decrement operators – overloading binary operators – overloading with friend functions – type conversion – one argument constructor and operator function – overloading stream operators.

Inheritance: Introduction – Types of Inheritance – Virtual base classes – constructors and destructors and inheritance – abstract classes – qualifier classes and inheritance – common constructor – pointers and inheritance – overloading member function.

PART A – 2 MARK QUESTIONS

1. Define constructor

A constructor is a special member function whose task is to initialize the objects of its class. It is special because its name is same as class name. The constructor is invoked whenever an object of its associated class is created. It is called constructor because it constructs the values of data members of the class

Eg:
integer Class
{

```
.....  
public:  
integer( );//constructo r  
.....  
}
```

2. What is meant by operator overloading?

C++ has the ability to provide the operators with a special meaning for a data type. This mechanism of giving such special meanings to an operator is known as Operator overloading. It provides a flexible option for the creation of new definitions for C++ operators.

3. What are the operators that cannot be overloaded?

- Class member access operator (. , .*)
- Scope resolution operator (::)
- Size operator (sizeof)
- Conditional operator (?:)

4. Define Inheritance.

Inheritance is the process by which objects of one class acquire the properties of objects of another class. It supports the concept of hierarchical classification. For example the bird 'Robin'

5. What is meant by multilevel inheritance?

If a class is derived from a class, which in turn is derived from another class, is called multilevel inheritance. This process can be extended to any number of levels.

Eg:

Base class Grand father

Intermediate

Base class Father

Derived class Child

6. List out the types of inheritance.

- Single Inheritance
- Multiple Inheritance

- Hierarchical Inheritance
- Multilevel Inheritance
- Hybrid Inheritance
- Multi-path Inheritance

7. What is Function Overloading?

A function with the same name performing different operations is called function overloading. To achieve function overloading, functions should be declared with the same name but different number and type of arguments.

8. List some of the special characteristics of constructor.

- Constructors should be declared in the public section.
- They are invoked automatically when the objects are created.
- They do not have return types
- They cannot be inherited.

9. Give the various types of constructors.

There are four types of constructors. They are,

- **Default constructors** – A constructor that accepts no parameters.
- **Parameterized constructors** – The constructors that can take arguments.
- **Copy constructor** – It takes a reference to an object of the same class as itself as an argument.
- **Dynamic constructors** – Used to allocate memory while creating objects.

10. Define Destructor.

A destructor is used to destroy the objects that have been created by a constructor. It is a special member function whose name is same as the class and is preceded by a tilde ‘~’ symbol. When an object goes out from object creation, automatically destructor will be executed.

Example:

```
class File {  
public:  
    ~File();    //destructor declaration  
};  
File::~~File()  
{  
    close();    // destructor definition  
}
```

11. List Some Of The Rules For Operator Overloading.

- Only existing operators can be overloaded.
- We cannot change the basic meaning of an operator.
- The overloaded operator must have at least one operand.
- Overloaded operators follow the syntax rules of the original operators.

12. Define parameterized constructor.

Constructor with arguments is called parameterized constructor.

Eg:

```
Class Student
{
    int m,n;
    Public:
    Student (int x, int y)
    {
        m=x; n=y;
    }
}
```

13. What are the characteristics of destructor?

- A destructor must be declared in the public section of a class.
- A class cannot have more than one destructor.
- It has no return type.
- It is incorrect to even declare a void return type

14. Define Copy Constructor

A copy constructor is used to declare and initialize an object from another object. It takes a reference to an object of the same class as an argument

Eg: integer i2 (i1);

Would define the object i2 at the same time initialize it to the values of i1.

15. Define explicit constructor.

Constructor can be defined explicitly by using the keyword “explicit” is known as an explicit constructor. The explicit constructor will be executed when we call the constructor explicitly.

Ex:

```
explicit brother (string name)
{
}
```

```
        Body of the explicit constructor  
    }
```

Note: brother is a class name.

16. List out the operators that cannot be overload as friend functions.

The operators that cannot be overload as friend function are:

- i. Assignment operator =
- ii. Function call operator ()
- iii. Array subscript operator []
- iv. Access to class member using pointer to object operator

What is the purpose of using operator function? Write its syntax.

To define an additional task to an operator, we must specify what it means in relation to the class to which the operator is applied. This is done by Operator function, which describes the task. Operator functions are either member functions or friend functions.

The general form is :

```
return type classname :: operator op(arglist )
```

```
{ // function body }
```

How an overloaded operator can be invoked using Friend functions?

- In case of unary operators,
overloaded operator can be invoked as Operator op (x);
- In case of binary operators,
overloaded operator can be invoked as Operator op (x , y)

17. Define abstract classes

An abstract class is one that is not used to create objects. An abstract class is designed only to act as a base class. It is a design concept in program development and provides a base upon which other classes may be built.

18. Write a program to overload insertion operator(<<) with friend function.

```
#include<iostream.h>  
#include<constream.h>  
class string  
{
```

```
        char *s;
        public:
        string(char *k)
        {
            s = k;
        }
friend ostream& operator << (ostream&put, string &k)
{
    put<<k.s;
    return put;
}
};
int main()
{
    clrscr();
    string s("INDIA");

    cout<<s;

    return 0;
}
```

OUTPUT

INDIA

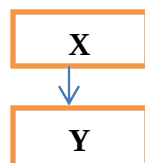
19. How an overloaded operator can be invoked using member functions?

- In case of Unary operators, overloaded operator can be invoked as op object_name or object_name op
- In case of binary operators, it would be invoked as Object . operator op(y)

where op is the overloaded operator and y is the argument.

20. What is meant by single inheritance?

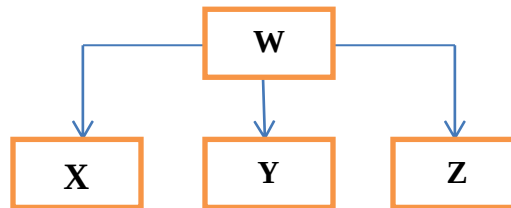
When only one base class is used for derivation of a class and the derived class is not used as base class, such type of inheritance between one base and derived class is known as single inheritance.



Here X is a base class. Y is a derived class. This type involves one base and derived class. Further, no class is derived from Y.

21. What is hierarchical inheritance?

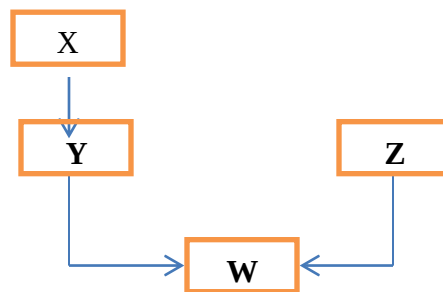
When a single base class is used for derivation of two or more classes, it is known as hierarchical inheritance.



W is only one base class. X, Y and Z are derived classes. Further, X, Y and Z are not used for deriving a class.

22. What is Hybrid Inheritance?

The combination of one or more type of inheritance is known as hybrid inheritance.

**23. List the rules for inheritance**

- 1) The default mode is private
- 2) Difficult to inherit the base class private members
- 3) The base class public and protected member functions are considered as public and protected member functions in derived class, When those class is publicly inherited from base class

PART-B(11 MARKS)**CONSTRUCTOR**

A constructor is a special function in a class that is called when a new object of the class is declared. It therefore provides the opportunity to initialize objects as they are created and to ensure that data members only contain valid values. It is a member function whose task is to

initialize the object of its class and allocate the required resources such as memory. It is distinct from other members of the class because it has the same name as its class.

A constructor can be defined declared and defined with the following syntax:

```
class classname
{
    .....;
    public :
        classname();
};
classname::classname()
{
    //Body of constructor
}
```

The diagram illustrates the syntax of a constructor with the following labels and arrows:

- Private members**: Points to the ellipsis (`.....;`) inside the class body.
- Constructor declaration**: Points to the `classname();` line within the `public :` section.
- Constructor definition**: Points to the `classname::classname()` line outside the class.
- //Body of constructor**: Points to the code inside the curly braces of the constructor definition.

Constructors have some special constraints or rules that must be followed while defining them. These are as follows:

- Constructor is executed automatically whenever the class is instantiated i.e., object of the class is created.
- It always has the same name as the class in which it is defined
- It cannot have any return type, not even void.
- It is normally used to initialize the data members of a class.
- It is also used to allocate resources like memory to the dynamic data members of a class.
- It is normally declared in the public access within the class.

For example, let us declare a class with class name 'circle'. If we use constructor, then the constructor name must also be 'circle'. The program can be written as follows:

```
/*Program 8.1: Program to demonstrate constructor while calculating the area of a circle*/
#include<iostream.h>
#include<conio.h>
#define PI 3.1415
class circle
{
private:
float
radius;
public:
circle(); //constructor declaration
void area() // member function
{
cout<<"\nArea of the circle is "<<PI*radius*radius<<" sq.units\n";
}
};
circle::circle() //constructor definition
{
```

```
cout<<"\nEnter radius of the circle: ";
```

```
cin>>radius;

}
int main()
{
clrscr();
circle c; //object created; constructor invoked automatically
c.area();
getch();
return 0;
}
```

The output of the above program will be :

Enter radius of the circle: 5

Area of the circle is 78.5375 sq. units

In the above program, the constructor circle() takes the value of radius from the keyboard.

Although the data member is private, the constructor could initialize them. The statement circle c ; declares a variable c as object of (type) class circle. It also calls the constructor implicitly.

The Default Constructors

The constructor which takes no arguments is called the default constructor. The following code fragment shows the syntax of a default constructor.

```
class class_name
```

```
{
private:
data members;
public:
class_name( ); //default constructor
};
class_name : : class_name( )
{
/* definition of constructor without any arguments and body */
}
```

If no constructor is defined for a class, then the compiler supplies the default constructor.

PARAMETERIZED CONSTRUCTOR

The constructor that can take arguments are called parameterized constructor. The arguments can be separated by commas and they can be specified within braces similar to the argument list in function.

When a constructor has been parameterized, the object declaration without parameter may not work. In case of parameterized constructor, we must provide the appropriate arguments to the constructor when an object is declared. This can be done in two ways:

- By calling the constructor explicitly.
- By calling the constructor implicitly.

The implicit call method is sometimes known as shorthand method as it is shorter and is easy to implement.

Let us consider the following example to demonstrate how parameterized constructor works.

// Program 8.2: Program creating parameterized constructor

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class student
```

```
{
```

```
private:
```

```
int roll,age, marks;
```

```
public:
```

```
student(int r, int m, int a); //parameterized constructor
```

```
void display( )
```

```
{
```

```
cout<<"\nRoll number : " <<roll <<endl;
```

```
cout<<"Total marks : " <<marks<<endl;
```

```
cout<<"Age:"<<age<<endl;
```

```
}
```

```
}; //end of class declaration
```

```
student : : student(int r, int m, int a) //constructor definition
```

```
{
```

```
roll = r;
```

```
marks =m;
```

```
age=a;
```

```
}
```

```
int main( )
```

```
{
```

```

                                Constructor called implicitly
                                /
student manoj(5,430,16);      //object creation
cout<<"\nData of student 1 : "<<endl;

manoj.display();

                                Constructor called explicitly
                                /
student rahul = student(6,380,15); //object creation
cout<<"\n\nData of student 2 : "<<endl;
rahul.display();
getch();
return 0;
}
```

The output of the above program will be like this:

```
Data of student 1 :
Roll number      : 5
Total Marks      : 430
Age              : 16
```

```
Data of student 2 :
Roll number : 6
Total Marks : 380
Age : 15
```

In the above program, the statement `student(int r, int m, int a, );` is a parameterized constructor with two arguments. In the `main( )` function, we see that there are two objects, `manoj` and `rahul` of class type `student`. The object `manoj` is initialized with the values 5,430 and 16 with the implicit call statement `student manoj(5, 430,16);`. The object `rahul` is initialized with the values 6,380 and 15 with the explicit call statement `student rahul = student(6, 380,15);` This statement creates a student object `rahul` and passes the values 6, 380 and 15 to it.

Constructors with Default Arguments

It is possible to have a constructor with arguments having default values. It means that if we have a parameterized constructor with n parameters, then we can invoke it with less than n parameters specified in the call.

It is useful when most of the objects to be created are likely to have the same value for some data members. We need not specify that in every invocation. For example, suppose we want to record the data of class `x` (ten) students. For this, we can consider the same class `student` as shown in Program 8.2. Most of the students are of age 16. Hence, their birth year is likely to be the same. Only few of them may have a different year of birth. For this we can use the statement `student(int r, int m, int a=16);`

where the third parameter is given the default value. When we write the following statement

student s(1,360);

it assigns the values 1 and 360 to the argument r and m respectively and 16 to the argument a by default. Again, if we write

student t(2,400,15);

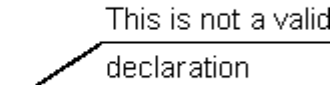
it assigns 2, 400 and 15 to r, m and a respectively. Thus, the actual arguments, when specified, overrides the default values.

COPY CONSTRUCTOR

Object of the same class cannot be passed as argument to constructor to that class by value method.

class student

```
{
    private:
        .....
    public:
        student(student s);
};
```

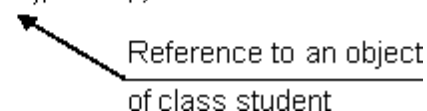


This is not a valid declaration

But it is possible to pass an object of the same class as argument to a constructor by reference method. Such type of constructor, i.e., a constructor having parameter which is a reference to object of the same class is called a copy constructor. Thus, the following declaration is a valid declaration.

class student

```
{
    private:
        int roll, marks, age;    //data members
    public:
        student(student &s);    //copy constructor
};
```



Reference to an object of class student

Copy constructor is also called one argument constructor as it takes only one argument. The main use of copy constructor is to initialize the objects while in creation, also used to copy an object. This constructor allows the programmer to create a new object from an existing one by initialization. Let us demonstrate copy constructor with the following program .:

//Program 8.3: Program to demonstrate copy constructor

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
class student
```

```
{
```

```
private:
int roll, marks, age;
public:
student(int r,int m,int a) //parameterized constructor
{
roll = r;
marks = m;
age = a;
}
student(student &s) //copy constructor
{
roll = s.roll;
marks = s.marks;
age=s.age;
}
void display( )
{
cout<<"Roll number :" <<roll <<endl;
cout<<"Total marks :"<<marks<<endl;
cout<<"Age:"<<age<<endl;
}
};
int main( )
{
```

```
clrscr();
student t(3,350,17); // or student k(t);
student k = t; // invokes copy constructor
cout<<"\\nData of student t:"<<endl;
t.display();
cout<<"\\n\\nData of student k:"<<endl;
k.display();
getch();
return 0;
}
```

The outout of the above program will be like this:

Data of student t :

Roll number : 3

Total marks : 350

Age : 17

Data of student k :

Roll number : 3

Total marks : 350

Age : 17

The statements
 student t(3,350,17); //invokes parameterized constructor
 student k = t; //invokes copy constructor
 initialize one object k with another object t. The data members of t are copied member by member into k. When we see the output of the program we observe that the data of both the objects t and k are same.

OVERLOADING OF CONSTRUCTORS

A class may contain multiple constructors i.e., a class can have more than one constructor with the same name. The constructors are then recognized depending on the arguments passed. It may differ in terms of arguments, or data types of their arguments, or both. This is called overloading of constructors or constructor overloading.

Program 8.3 is also an example of constructor overloading as it has two constructors: one is parameterized and the other is a copy constructor. Let us take a suitable example to demonstrate overloading of constructors.

//Program 8.4: Demonstration of constructor overloading

```
#include<iostream.h>
#include<math.h>
#include<conio.h>
class complex
{
private:
float real,imag;
public:
complex( ) //constructor with no argument
{
real = imag = 0.0;
}
complex(float r, float i) //constructor with two arguments
{
real = r;
imag = i;
}
complex(complex &c) //copy constructor
{
real = c.real;
imag = c.imag;
}
complex addition(complex d); /*member function returning
object and taking object as argument */
void display( ) //Display member function

{
cout<<real;
if(imag < 0)

cout<<"-i";
```

```
else
cout<<"i";
cout<<fabs(imag); /*fabs calcute the absolute value
of a floating point number */
}
};
complex complex::addition(complex d)
{
complex temp; //temporary object of type complex class
temp.real=real+d.real; //real parts added
temp.imag=imag+d.imag; //imaginary parts added
return(temp);
}
int main( )
{
clrscr( );
complex x1,x4; //invokes default constructor
cout<<"\nThe complex numbers in a+ib form :\n\n";
cout<<"First complex number: ";
x1.display();
complex x2(1.5,5.3); //invokes parameterized constructor
cout<<"\nSecond complex number: ";
x2.display();
complex x3(2.4,1.9);
cout<<"\nThird complex number: ";
x3.display();
cout<<"\nAddition of second and third complex number: ";
x4=x2.addition(x3); //function call
x4.display( );
cout<<"\nThe result is copied to another object: ";
complex x5(x4); //invokes copy constructor
x5.display();
getch();
return 0;
}
```

The output of the above program will be:

The complex numbers in a+ib form:

First complex number: 0+i0

Second complex number: 1.5+i5.3

Third complex number: 2.4+i1.9

Addition of second and third complex number: 3.9+i7.2

The result is copied to another object: 3.9+i7.2

We have the following three constructors

complex();

complex(float r, float i) ;

and complex(complex &c);

in the above program which indicates overloading of constructors. These constructors are invoked during the creation of an object depending on the number and types of arguments passed. The default constructor `complex( )`; initializes the data members `real` and `imag` to 0.0. In `main( )`, the statement `complex x2 (1.5,5.3)`; passes two parameters to the constructor explicitly with the help of the parameterized constructor `complex (float r, float i)`; With the help of copy constructor `complex (complex &c)`; data members of one object is copied member by member into another.

DESTRUCTOR

Like constructors, destructors are also special member functions used in C++ programming language. Destructors have the opposite function of a constructor. The main function of destructors is to free memory and to release resources. Destructors take the same name as that of the class name preceded by a tilde ~. A destructor takes no arguments and has no return type.

The general syntax of a destructor is as follows:

```
class classname
{
    private :           Private members
        ..... ;

    public :            Destructor declaration
        ~classname( );

};

Destructor definition
classname :: ~classname( )
{
    //Body of destructor
}
```

In the above, the symbol tilde ~ represents a destructor which precedes the name of the class. Like the default constructor, the compiler always creates a default destructor if we don't create one. Similar to constructors, a destructor must be declared in the public section of a class. Destructors cannot be overloaded i.e., a class cannot have more than one destructor.

/* Program 8.5 : Program demonstrating destructor */

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
class student
{
private:
int age;
char name[25];
public:
student(int a, char n[25])
{
age=a;
strcpy(name,n);
}
```

```
void show()
{
cout<<"\nThe name of the student is: "<<name;
cout<<"\nHis age is: "<<age;
}
~student() //destructor defined
{
cout<<"\nObject Destroyed";
}
};
int main()
{
student s1(21,"Rahul");
student s2(23,"Dipankar");
clrscr();
s1.show();
s2.show();
return 0;
}
```

To see the execution of destructor, we have to press Alt+F5 after compiling and running the program. The output will be like this:

The name of the student is : Rahul

His age is : 21

The name of the student is : Dipankar

His age is : 23

Object Destroyed

Object Destroyed

The following points should be kept in mind while defining and writing the syntax for the destructor

- A destructor must be declared with the same name as that of the class to which it belongs. But destructor name should be preceded by a tilde (~).
- A destructor should be declared with no return type.
- A destructor must have public access in the class declaration.

DYNAMIC INITIALIZATION OF OBJECTS

We can dynamically initialize objects through constructors. Object's data members can be initialized dynamically at run time even after their creation.

//Program 8.6 Demonstration of dynamic initialization of object

```
#include<iostream.h>
#include<conio.h>
#include<math.h>
class number
{
public:
int num;
number(int n)
{
num=n;
}
int sum( )
{
num=num+5;
return(num);
}
};
int main( )
{
number obj1(1); // parameterized constructor invoked
number obj2(2);
clrscr();
cout<<"\nValue of object 1 and object 2 are : ";
cout<<obj1.num<<"\t"<<obj2.num;
number obj3(obj1.sum( )); //dynamic initialization of object
cout<<"\nValue of object 1 after calling sum() : "<<obj1.num;
cout<<"\nValue of object 3 is : "<<obj3.num;
getch();
return 0;
}
```

The output of the above program will be like this:

Value of object 1 and object 2 are : 1 2

Value of object 1 after calling sum() : 6

Value of object 3 is : 6

The statements `number obj1(1);` and `number obj2(2);` will initialize `obj1` and `obj2` by 1 and 2 respectively by invoking the constructor `number(int n)`. `obj1.sum( )` will return the value 6. This is passed as argument in the statement

`number obj3(obj1.sum( ));`

where `obj3` is initialized dynamically with the value returned by `obj1.sum( )`.

Like normal and member functions, constructors also support recursion. The program given next explains this.

20 Write a program to invoke constructor recursively and calculate the triangular number of the entered number.

```
# include <iostream.h>
# include <conio.h>
# include <process.h>

class tri_num
{
    int f;
    public :

    tri_num( ) { f=0; }

    void sum( int j) { f=f+j; }

    tri_num(int m)
    {
        if (m==0)
        {
            cout <<"Triangular number : "<<f;
            exit(1);
        }

        sum(m);
        tri_num::tri_num(--m);
    }

};

void main( )
{
    clrscr( );

    tri_num a;
    a.tri_num::tri_num(5);
}
```

OUTPUT

Triangular number : 15

Explanation: In the above program, class tri_num is declared with private integer f and member function sum(). The class also has zero-argument constructor and one-argument constructor. In function main(), a is an object of tri_num class. When object a is declared, the zero-argument constructor is executed. The object a invokes the constructor explicitly and passes integer value to it. The passed value is received by the variable m of the constructor. The if statement checks the value of variable m and if it is zero the triangular number is displayed and program is terminated.

The sum() function is called by the constructor and cumulative sum is calculated and stored in the member variable *f*. Followed by the call of function sum(), the constructor tri_num() is executed and the value of m is decreased by one. Thus, recursion takes places and each time value of m decreases. When the value of m becomes zero, the program terminates. We get the triangular number.

Overloading Functions:

Operator overloading is an important and useful feature of C++. The concept of operator overloading is quite similar to that of function overloading.

An operator is a symbol that indicates an operation. It is used to perform operation with constants and variables. Without an operator, programmer cannot built an expression.

C++ frequently uses user-defined data types, which is a combination of one or more basic data types. C++ has an ability to treat user-defined data type like the one they were built-in type.

User-defined data types created from class or struct are nothing but combination of one or more variables of basic data types. The compiler knows how to perform various operations using operators for built-in types; however, for the objects those are instance of the class, the operation routine must be defined by the programmer.

Write a program to perform addition of two objects and store the result in third object. Display contents of all the three objects.

```
# include <iostream.h>
# include <constream.h>
```

```
class number
```

```
{
    public:
    int x;
    int y;
```

```
    number( ) { } // ZERO ARGUMENT CONSTRUCTOR
```

```
    number (int j, int k ) // TWO ARGUMENT CONSTRUCTOR
```

```
    {
        x=j;
        y=k;
    }
```

```
void show( )
```

```
{
    cout <<"\n x="<<x <<" y="<<y;
}
```

```
};
```

```
void main( )
```

```
{
    clrscr ( );
    number A(2,3), B(4,5), C ;
```

```
A.show ( );  
B.show ( );  
  
C.x=A.x+B.x; // ADDITION OF TWO OBJECTS  
C.y=A.y+B.y; // USING MEMBER VARIABLES DIRECTLY  
  
C.show( );  
}
```

OUTPUT

x=2 y=3

x=4 y=5

x=6 y=8

Explanation: In the above program, A, B and C are objects of class number. Using constructor, objects are initialized. Consider the following statements:

C.x=A.x+B.x;

C.y=A.y+B.y;

OPERATOR OVERLOADING

The capability to relate the existing operator with a member function and use the resulting operator with objects of its class as its operands is called operator overloading.

THE KEYWORD OPERATOR

The keyword operator defines a new action or operation to the operator.

Syntax:

Return type operator operator symbol (parameters)

```
{  
    statement1;  
    statement2;  
}
```

The keyword ‘operator’, followed by an operator symbol, defines a new (overloaded) action of the given operator.

Example:

number operator + (number D)

```
{  
    number T;  
    T.x=x+D.x;  
    T.y=y+D.y;  
    return T;  
}
```

Overloaded operators are redefined within a C++ class using the keyword operator followed by an operator symbol. When an operator is overloaded, the produced symbol is called the operator function name. The above declarations provide an extra meaning to the operator. Operator functions should be either member functions or friend functions. A friend function requires one argument for unary operators and two for binary operators. The member function requires one argument for binary operators and no argument for unary operators. When the member function is called, the calling object is passed implicitly to the function and hence available for member

function. While using friend functions, it is essential to pass the objects by value or reference.

The prototype of operator functions in classes can be written as follows:

- (a) void operator ++();
- (b) void operator --();
- (c) void operator - ();
- (d) num operator+(num);
- (e) friend num operator * (int ,num);
- (f) void operator = (num);

Operator overloading can be carried out in the following steps:

- (a) Define a class which is to be used with overloading operations.
- (b) The public declaration section of the class should contain the prototype of the function operator().
- (c) Define the definition of the operator () function with proper operations for which it is declared.

8.2 Write a program to perform addition of two objects using operator keyword.

```
# include <iostream.h>
# include <constream.h>
```

```
class number
{
```

```
    public:
    int x;
    int y;
```

```
    number( ) { } // ZERO ARGUMENT CONSTRUCTOR
```

```
    number(int j, int k ) // TWO ARGUMENT CONSTRUCTOR
```

```
    {
        x=j;
        y=k;
    }
```

```
    number operator + ( number D)
```

```
    {
        number T;
        T.x=x+D.x;
        T.y=y+D.y;
        return T;
    }
```

```

void show( )
{
    cout <<"\n x="<<x <<" y="<<y;
}

};
void main( )

{
    clrscr ( )
    number A(2,3),B(4,5),C;
    A.show ( );
    B.show ( );
    C=A+B;
    C.show( );
}

```

OUTPUT

x=2 y=3

x=4 y=5

x=6 y=8

Explanation: In the above program, A, B and C are objects of class number. Here, the addition has been performed using statement C=A+B. Remember, in the last program we were not able to execute this statement. Instead of this, two separate statements were used to perform addition. In this program, the statements that perform addition operation of each individual member of objects are written in function operator. The operator has return type and single argument. It also uses a local object (T) to hold addition as long as the operator function is active. Whenever the statement C=A+B is executed, the compiler searches for definition of operator +. The object A invokes the operator function and object B is passed as argument. The copy of object B is stored in the formal argument D. The member variables of A are directly available in operator function as the function is invoked by the same object. The addition of individual members are carried out and stored in member variable of object T. The return type of operator function is same as that of its class. The function returns object T and it is assigned to variable C.

Overloading unary operators

Overloading devoid of explicit argument to an operator function is called as unary operator overloading. The operator ++, --, and – are unary operators. The unary operators ++ and -- can be used as **prefix** or **suffix** with the functions. These operators have only single operand. The examples given below illustrate the overloading of unary operators.

8.3 Write a program to increment member variables of object. Overload unary ++ operator.

```

# include <iostream.h>
# include <constream.h>

```

```

class num
{

```

```

private :
int a,b,c,d;

public :

num ( int j, int k, int m ,int l)
{
a=j;
b=k;
c=m;
d=l;
}

void show(void);
void operator ++( );

};

void num :: show( )
{
cout <<" A= "<<a <<" B= " <<b <<" C = "<<c <<" D = "<<d;
}

void num :: operator ++( )
{
++a; ++b; ++c; ++d;
}

main( )
{
clrscr( );
num X(3,2,5,7);
cout <<"\n Before Increment of X : ";
X.show( );
++X;
cout <<"\n After Increment of X : ";
X.show( );
return 0;
}

```

OUTPUT**Before Increment of X : A= 3 B= 2 C = 5 D = 7****After Increment of X : A= 4 B= 3 C = 6 D = 8**

Explanation: In the above example, the class num contains four integer variables a, b, c and d. The class also has two-member functions show() and operator ++() and one parameterized constructor. The constructor is used to initialize object. The show() displays the contents of the member variables. The operator ++() overloads the unary operator ++. When this operator is

used with integer or float variables, its value is increased by one. In this function, ++ operator precedes each member variable of class. This operation increments the value of each variable by one.

In function main(), the statement ++X calls the function operator ++(), where, X is an object of the class num. The function can also be called using statement X.operator ++(). In the output, values of member variables before and after increment operations are displayed.

8.4 Write a program to overload – operator.

```
# include <iostream.h>
# include <conio.h>
class num
{
    private :
    int a,b,c,d;
    public :
    num ( int x, int y, int z, int w)
    {
        a=x;
        b=y;
        c=z;
        d=w;
    }
    void show(void);
    void operator -( );
};
void num :: show( )
{
    cout <<"A= "<<a <<" B= " <<b <<" C = "<<c <<" D = "<<d;
}
void num :: operator -( )
{
    a=-a;
    b=-b;

    c=-c;
    d=-d;
}
main( )
{
    clrscr( );
    num X(2,2,8,4);

    cout <<"\nBefore Negation of X : ";
    X.show( );
    -X;
    cout <<"\nAfter Negation of X : ";
    X.show( );
```

```
    return 0;
}
```

OUTPUT**Before Negation of X : A= 2 B= 2 C = 8 D = 4****After Negation of X : A= -2 B= -2 C = -8 D = -4**

Explanation: The above program is same as the previous one. Here, the operator – is overloaded. The statement –X calls the function operator –(). The function operator –() makes all the member variables negative. The function show() displays the values of member variables on the screen.

constraint on increment and decrement operators

When an operator (increment/decrement) is used as prefix with object, its value is incremented/decremented before operation and on the other hand the postfix use of operator increments/decrements the value of variable after its use.

When ++ and -- operators are overloaded, no difference exists between postfix and prefix overloaded operator functions. The system has no way of determining whether the operators are overloaded for postfix or prefix operation. Hence, the operator must be overloaded in such a way that it will work for both prefix and postfix operations. The ++ or -- operator overloaded for prefix operation works for both prefix as well as postfix operations but with a warning message, but not vice-versa. To make a distinction between prefix and postfix notation of operator, a new syntax is used to indicate postfix operator overloading function. The syntaxes are as follows:

Operator ++(int) // postfix notation

Operator ++() // prefix notation

The argument followed by operator (++ or --) should have type 'int'. When a postfix operator ++ or operator -- is declared, the last parameter must be declared with the type int. No other types such as float, long etc., are allowed. We can use this operator with all types of variables including float, long etc. Declaring int does not mean that it is only for integer type. The following program illustrates overloading of ++ operator in postfix and prefix style.

8.6 Write a program to overload ++ and -- operator for prefix and postfix use.

```
# include <iostream.h>
```

```
# include <constream.h>
```

```
class number
```

```
{
```

```
    float x;
```

```
    public :
```

```
    number ( float k) { x=k; }
```

```
void operator ++ (int) // postfix notation
```

```
{ x++; }
```

```
void operator -- ( ) // prefix notation
```

```
{ --x; }
```

```
void show( ) { cout <<"\n x="<<x; }
```

```
};  
  
void main( )  
{  
    clrscr( );  
    number N(2.3);  
    cout <<"\n Before Incrimination: ";  
    N.show ( );  
    cout <<"\n After Incrimination: ";  
    N++; // postfix increment  
    N.show( );  
    cout <<"\n After Decrementation:";  
    --N; // prefix decrement  
    N.show( );  
}
```

OUTPUT**Before incrimination:****x=2.3****After incrimination:****x=3.3****After decrementation:****x=2.3**

Explanation: In this program, operator ++ and -- are overloaded. The ++ operator is overloaded for postfix use and -- operator is overloaded for prefix use. You can see the keyword (int) is followed by the operator ++, which is necessary for postfix notation of operator. The operator -- is overloaded for prefix operation. Here, a value of float member variable is incremented and decremented.

overloading binary operators

Overloading with a single parameter is called as binary operator overloading. Like unary operators, binary operator can also be overloaded. Binary operators require two operands. Binary operators are overloaded by using member functions and friend functions.

(1) OVERLOADING BINARY OPERATORS USING MEMBER FUNCTIONS

If overloaded as a member function they require one argument. The argument contains value of the object, which is to the right of the operator. If we want to perform the addition of two objects o1 and o2, the overloading function should be declared as follows:

```
operator(num o2);
```

Where, num is a class name and o2 is an object.

To call function operator the statement is as follows:

```
o3=o1+o2;
```

We know that a member function can be called by using class of that object. Hence, the called member function is always preceded by the object. Here, in the above statement, the object o1 invokes the function operator() and the object o2 is used as an argument for the function. The above statement can also be written as follows:

```
o3=o1.operator +(o2);
```

Here, the data members of *o1* are passed directly and data members of *o2* are passed as an argument. While overloading binary operators, the left-hand operand calls the operator function and right-hand operand is used as an argument.

② OVERLOADING BINARY OPERATORS USING FRIEND FUNCTIONS

The friend can be used alternatively with member functions for overloading of binary operators. The friend function requires two operands to be passed as arguments.

```
o3=o1+o2;
```

```
o3=operator +(o1,o2);
```

Both the above statements have the same meaning. In the second statement, two objects are passed to the operator function.

The use of member function and friend function produces the same result. Friend functions are useful when we require performing an operation with operand of two different types. Consider the statements:

```
X =Y+3;
```

```
X =3+Y;
```

Where, X and Y are objects of same type. The first statement is valid. However, the second statement will not work. The first operand must be an object of the same class. This problem can be overcome by using friend function. The friend function can be called without using object. The friend function can be used with standard data type as left-hand operand and with an object as right-hand operand.

Following programs are illustrated based on the above discussion.

8.7 Write a program to overload + binary operator.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```
class num
{
    private :
        int a,b,c,d;

    public :

        void input(void);
        void show(void);
        num operator+(num);

};
```

```
void num :: input( )
{
    cout <<"\n Enter Values for a,b,c and d : ";
    cin >>a>> b>>c>>d;
}
```

```

void num :: show( )
{
    cout <<" A= "<<a <<" B= " <<b <<" C = "<<c <<" D = "<<d <<"\n";
}

num num :: operator +(num t)

{
    num tmp;

    tmp.a=a+t.a;
    tmp.b=b+t.b;
    tmp.c=c+t.c;
    tmp.d=d+t.d;
    return (tmp);
}

main( )
{
    clrscr( );
    num X,Y,Z;
    cout <<"\n Object X";
    X.input( );
    cout <<"\n Object Y";
    Y.input( );
    Z=X+Y;
    cout <<"\nX : ";
    X.show();
    cout <<"Y : ";
    Y.show( );
    cout <<"Z : ";

    Z.show( );
    return 0;
}

```

OUTPUT**Object X****Enter Values for a,b,c and d : 1 4 2 1****Object Y****Enter Values for a,b,c and d : 2 5 4 2****X : A= 1 B= 4 C = 2 D = 1****Y : A= 2 B= 5 C = 4 D = 2****Z : A= 3 B= 9 C = 6 D = 3**

Explanation: In the above program, binary operator + is overloaded. Using the overloading operator +, addition of member variables of two objects are performed and results are assigned to

member variables of third object. In this program X, Y and Z are objects of class num. The statement $Z=X+Y$ invokes the operator function. In this statement the object y is assigned to object t of operator function and member variables of X are accessed directly. The object tmp is used for holding the result of addition and it is returned to object Z after function execution. The function show() displays the values of three objects.

8.8 Write a program to perform multiplication using an integer and object.

Usefriend function.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```
class num
```

```
{
```

```
private :
```

```
int a,b,c,d;
```

```
public :
```

```
void input(void);
```

```
void show(void);
```

```
friend num operator * (int ,num); // friend function declaration
```

```
};
```

```
void num :: input( )
```

```
{
```

```
cout <<"\n Enter Values for a,b,c and d : ";
```

```
cin >>a>> b>>c>>d;
```

```
}
```

```
void num :: show( )
```

```
{
```

```
cout <<" A= "<<a <<" B= " <<b <<" C = "<<c <<" D = "<<d <<"\n";
```

```
}
```

```
num operator * (int a, num t)
```

```
{
```

```
num tmp;
```

```
tmp.a=a*t.a;
```

```
tmp.b=a*t.b;
```

```
tmp.c=a*t.c;
```

```
tmp.d=a*t.d;
```

```
return (tmp);
```

```
}
```

```
main( )
```

```

{
    clrscr( );
    num X,Z;
    cout <<"\n Object X";
    X.input( );
    Z=3*X;
    cout <<"\nX : ";
    X.show( );
    cout <<"Z : ";
    Z.show( );

```

```

return 0;
}

```

OUTPUT

Object X

Enter Values for a,b,c and d : 1 2 2 3

X : A= 1 B= 2 C = 2 D = 3

Z : A= 3 B= 6 C = 6 D = 9

Explanation: In the above program, the equation $Z=3*X$ contains integer and class object. We know that the left-hand operand is always used to invoke the function and the right-hand operand is passed as an argument. In such type of equations member functions are not useful because the left-hand operand is integer and cannot invoke the function. Hence, the function operator $*$ () is declared as friend. The friend function calls the operator $*$ () and carries the multiplication of each member variable by three. The results are displayed in the output.

overloading with friend functions

Friend functions are more useful in operator overloading. They offer better flexibility which is not provided by the member function of the class. The difference between member function and friend function is that the member function takes arguments explicitly. Quite the opposite, the friend function needs the parameters to be explicitly passed. The syntax of operator overloading with friend function is as follows:

friend return-type operator operator-symbol(variable1, variable2)

```

{
    statement1;
    statement2;
}

```

The keyword friend precedes function prototype declaration. It must be written inside the class. The function can be defined inside or outside the class. The arguments used in friend functions are generally objects of the friend classes. A friend function is similar to normal function. The only difference is that friend function can access private members of the class through the objects. Friend function has no permission to access private members of a class directly. However, it can access the private members via objects of the same class.

8.9 Write a program to overload unary operator using friend function.

```

# include <iostream.h>
# include <constream.h>

```

```
class complex
{
    float real,imag;
public:

complex( ) // zero argument constructor
{
    real=imag=0;
}

complex (float r, float i) // two argument constructor
{
    real=r;
    imag=i;
}

friend complex operator - ( complex c)
{
    c.real=-c.real;
    c.imag=-c.imag;
    return c;
}

void display( )
{
    cout <<"\n Real : "<<real;
    cout <<"\n Imag : "<<imag;
}
};

void main( )
{
    clrscr( );
    complex c1(1.5,2.5),c2;
    c1.display( );
    c2=-c1 ;
    cout <<"\n\n After Negation \n";
    c2.display( );
}
```

OUTPUT**Real : 1.5****Imag : 2.5****After Negation****Real : -1.5****Imag : -2.5**

Explanation: In the above program, operator -- is overloaded using friend function. The operator function is defined as friend. The statement c2=-c1 invokes the operator function. This statement also returns the negated values of c1 without affecting actual value of c1 and assigns it to object c2.

The negation operation can also be used with an object to alter its own data member variables. In such a case the object itself acts as a source and destination object. This can be accomplished by sending reference of object. Program 8.10 illustrates this.

8.10 Write a program to pass reference of an object to operator function.

```
# include <iostream.h>
# include <constream.h>

class complex
{
    float real,imag;
    public:
    complex (float r, float i) // two argument constructor
    {
        real=r;
        imag=i;
    }

    friend void operator - ( complex & c)
    {
        c.real=-c.real;
        c.imag=-c.imag;
    }

    void display( )
    {
        cout <<"\n Real : "<<real;
        cout <<"\n Imag : "<<imag;
    }
};

void main( )
{
    clrscr( );
    complex c1(1.5,2.5);
    c1.display( );
    -c1;
    cout <<"\n\n After Negation \n";
    c1.display( );
}
```

OUTPUT**Real : 1.5****Imag : 2.5****After Negation****Real : -1.5****Imag : -2.5**

Explanation: In the above program, the object c1 itself acts as source and destination object. The reference of object is passed to operator function. The object c is a reference object of c1. The values of object c are replaced by itself by applying negation.

type conversion

When constants and variables of various data types are clubbed in a single expression, automatic type conversion takes place. This is so for basic data types. The compiler has no idea about the user-defined data types and about their conversion to other data types. The programmer should write the routines that convert basic data types to user-defined data types or vice versa. There are three possibilities of data conversion as given below:

- (1) Conversion from basic data type to user-defined data type (class type).
- (2) Conversion from class type to basic data type.
- (3) Conversion from one class type to another class type.

(1) CONVERSION FROM BASIC TO CLASS TYPE

The conversion from basic to class type is easily carried out. It is automatically done by the compiler with the help of in-built routines or by applying typecasting. In this type the left-hand operand of = sign is always class type and right-hand operand is always basic type. The program given below explains the conversion from basic to class type.

8.11 Write a program to define constructor with no argument and with float argument. Explain how compiler invokes constructor depending on data type.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```
class data
```

```
{ int x;
```

```
float f;
```

```
public :
```

```
data ( )
```

```
{
```

```
    x=0;
```

```
    f=0;
```

```
}
```

```
data ( float m)
```

```
{
```

```
    x=2;
```

```
    f=m;
```

```
}
```

```

void show( )
{
    cout <<"\n x= "<<x <<" f = "<<f;
    cout <<"\n x= "<<x <<" f = "<<f;
}
};

int main( )
{
    clrscr( );
    data z;
    z=1;
    z.show( );
    z=2.5;
    z.show( );
    return 0;
}

```

OUTPUT

```

x= 2 f = 1
x= 2 f = 1
x= 2 f = 2.5
x= 2 f = 2.5

```

Explanation: In the above program, the class data has two member variables each of integer and float types respectively. It also has two constructors one with no argument and second with float argument. The member function show() displays the contents of the data members. In function main(), z is an object of class data. When z is created the constructor with no argument is called and data members are initialized to zero. When z is initialized to one the constructor with float argument is invoked. The integer value is converted to float type and assigned member variable f. Again when z is assigned to 2.5, same process is repeated. Thus, the conversion from basic to class type is carried out.

(2) CONVERSION FROM CLASS TYPE TO BASIC DATA TYPE

In the previous example, we studied how compiler makes conversion from basic to class type. The compiler does not have any knowledge about the user-defined data type built using classes. In this type of conversion, the programmer explicitly needs to tell the compiler how to perform conversion from class to basic type. These instructions are written in a member function. Such type of conversion is also known as overloading of type cast operators. The compiler first searches for the operator keyword followed by data type and if it is not defined, it applies the conversion functions. In this type, the left-hand operand is always of basic data type and right-hand operand is always of class type. While carrying this conversion, the statement should satisfy the following conditions:

- (1) The conversion function should not have any argument.
- (2) Do not mention return type.
- (3) It should be a class member function.

8.12 Write a program to convert class type data to basic type data.

```

# include <iostream.h>
# include <conio.h>

```

```
class data
{
    int x;
    float f;

public :

    data( )
    {
        x=0;
        f=0;
    }
    operator int( )
    {
        return (x);
    }

    operator float( )
    { return f; }

data ( float m)
{

    x=2;
    f=m;
}

void show( )
{
    cout <<"\n x= "<<x <<" f = "<<f;
    cout <<"\n x= "<<x <<" f = "<<f;
}

};

int main( )
{
    clrscr( );
    int j;
    float f;
    data a;
    a=5.5;

    j=a; // operator int ( ) is executed
    f=a; // operator float ( ) is executed

    cout <<"\n Value of j : "<<j;
```

```

    cout <<"\n Value of f : "<<f;
    return 0;
}

```

OUTPUT

Value of j : 2

Value of f : 5.5

Explanation: In the above program, the class data has two member variables each of integer and float data type. It also contains constructors as per described in the last example. In addition, it contains overloaded data types int and float. These functions are useful for conversion of data from class type to basic type. Consider the following statements:

(a) j=a;

(b) f=a;

In the first statement object a is assigned to integer variable j. We know that class type data is a combination of one or more basic data types. The class contains two-member functions operator int() and operator float(). Both these functions are able to convert data types from class to basic. In statement (a) the variable j is of integer type, the function operator int() is invoked and integer value data member is returned. In statement (b), f is of float type, the member function operator float() is invoked.

(3) CONVERSION FROM ONE CLASS TYPE TO ANOTHER CLASS TYPE

When an object of one class is assigned to object of another class, it is necessary to give clear-cut instructions to the compiler. How to make conversion between these two user-defined data types? The method must be instructed to the compiler. There are two ways to convert object data type from one class to another. One is to define a conversion operator function in source class or a one-argument constructor in a destination class. Consider the following example:

X=A;

Here, X is an object of class XYZ and A is an object of class ABC. The class ABC data type is converted to class XYZ. The conversion happens from class ABC to XYZ. The ABC is a source class and XYZ is a destination class.

We know the operator function operator data-type(). Here, data type may be built-in data type or user-defined data type. In the above declaration, the data type indicates target type of object.

Here, the conversion takes place from class ABC (source class) to class XYZ (destination class).

8.13 Write a program to convert integer to date and vice versa using conversion function in source class.

```

#include <iostream.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h>

```

```

class date
{
    char d[10];
public :
    date( ) { d[0]=NULL; }

```

```
date(char *e) { strcpy(d,e); }

void show( ) { cout <<d; }
};

class dmy
{
    int mt,dy,yr;

    public:

dmy ( ) { mt=dy=yr=0; }

dmy (int m, int d, int y)
{
    mt=m;
    dy=d;
    yr=y;
}
operator date( )
{
    char tmp[3],dt[9];

    itoa(dy,dt,10);
    strcat(dt,"-");

    itoa(mt,tmp,10);

    strcat(dt,tmp);
    strcat(dt,"-");

    itoa(yr,tmp,10);
    strcat(dt,tmp);
    return (date(dt));
}

void show( )
{
    cout <<dy<<" "<<mt<<" "<<yr;
}
};

int main( )
```

```
{
clrscr( );
date D1;
dmy D2(1,7,99);

D1=D2;
cout <<endl<<"D1=";
D1.show( );
cout <<endl<<"D2=";
D2.show( );
return 0;
}
```

OUTPUT

D1=7-1-99

D2=7 1 99

Explanation: In the above program, date and dmy are two classes declared. In function main(), D1 is an object of class date and D2 is an object of class dmy. The object D2 is initialized. The statement D1=D2 initializes D1 with D2. Here, both the objects D1 and D2 are of different types hence the conversion function date() is called to perform the conversion from one object to another object.

8.14 Write a program to convert integer to date and vice versa using conversion function in destination class.

```
# include <iostream.h>
# include <conio.h>
# include <stdlib.h>
# include <string.h>

class dmy
{
    int d,m,y;

    public :

    dmy( ) { d=m=y=0; }

    dmy(int da, int ma, int ya) { d=da; m=ma; y=ya; }

    int day( ) { return (d); }

    int month( ) { return (m); }

    int year( ) { return (y); }

    void show( ) { cout <<d<<" "<<m<<" "<<y; }
};
```

```
class date
{
    private :
        char dts[9];
    public :
        date( ) { dts[0]=0; }

    date ( char *e) { strcpy (dts,e); }

    void show( ) { cout <<dts; }

    date ( dmy k)
    {
        int d=k.day( ); // first member function
        int m=k.month( ); // second member function
        int y=k.year( ); // third member function

        char tmp[3];
        itoa(d,dts,10);
        strcat(dts,"-");
        itoa(m,tmp,10);
        strcat(dts,tmp);
        strcat(dts,"-");
        itoa(y,tmp,10);
        strcat(dts,tmp);
    }

};

int main()
{
    clrscr( );
    date D1;
    dmy D2(1,3,77);
    D1=D2;
    cout<<endl<<"D1=";

    D1.show( );
    cout <<endl<<"D2=";
    D2.show( );
    return 0;
}
OUTPUT
D1=1-3-77
D2=1 3 77
```

Explanation: In the above program, as soon as the statement D1=D2 is executed, the one-argument constructor defined in the class date is invoked. The one argument constructor carries the conversion. The constructor calls three-member function of dmy class to get the day, month, and year of the date. Here, in this program conversion is done by using the constructor in destination object.

8.15 Write a program to convert class data type to another class data type.

```
# include <iostream.h>
# include <conio.h>
class minutes
{
    int m;

public :

    minutes( )

    { m=240; }

    get( )
    { return (m);}

    void show( )
    { cout <<"\n Minutes = "<<m; }

};

class hours
{
    int h;
public :
    void operator = (minutes x);
    void show ( )
    { cout <<"\n Hours = "<<h; }

};

void hours:: operator = (minutes x)
{
    h=x.get( )/60;
}

int main( )
```

```

{
    clrscr( );
    minutes minute;
    hours hour;
    hour=minute;
    minute.show( );
    hour.show ( );
return 0;
}

```

OUTPUT

Minutes = 240

Hours = 4

Explanation: In the above program, two classes are declared. The class minutes has one integer member variable m and two member functions get() and show(). It also contains constructor without argument. The class hours has one integer member variable and show() member function. The class hours contains overloaded operator function. In function main(), minute is an object of class minutes and hour is an object of class hours. The program converts minutes to hours. The equation hour=minute invokes the operator function. In function operator(), x is an object of class minutes. The object x invokes the member function get() that returns total number of minutes. Number of hours is obtained by dividing the total number of minutes by 60. The equation h=x.get()/60 performs this task and assigns result to h. Thus, the result of the program is as per given above.

8.9 RULES FOR OVERLOADING OPERATORS

- Overloading of an operator cannot change the basic idea of an operator. When an operator is overloaded, its properties like syntax, precedence, and associativity remain constant. For example A and B are objects. The following statement

A+=B;

assigns addition of objects A and B to A. The overloaded operator must carry the same task like original operator according to the language. The following statement must perform the same operation like the last statement.

A=A+B;

- Overloading of an operator must never change its natural meaning. An overloaded operator + can be used for subtraction of two objects, but this type of code decreases the utility of the program. Remember that the aim of operator overloading is to comfort the programmer to carry various operations with objects. Consider the following program.

8.16 Misuse of operator overloading. Perform subtraction using + operator.

```

# include <iostream.h>
# include <constream.h>

```

```

class num
{
    int x;
    public:

    num( ) { x=0; }
    num( int k) { x=k; }

```

```

num operator + ( num n)
{
    num s;
    s.x=x-n.x;
    return s;
}

void show( ) { cout <<"\n x="<<x; }

};

void main( )
{
    clrscr( );
    num a(10), b(5),c;
    c=a+b;
    c.show( );
}
OUTPUT
X=5

```

Explanation: In the above program, the operator + is overloaded. It performs subtraction. Such type of misuse must be avoided while overloading operators. The programmer thought that it would perform addition but in reality, it performs subtraction.

- The overloaded operator should contain one operand of user-defined data type. Overloading operators are only for classes. We cannot overload the operator for built-in data types.
- Overloaded operators have the same syntax as the original operator. They cannot be prevailing over the original operators.
- There is no higher limit to the number of overloading for any operator. An operator can be overloaded for a number of times if the arguments are different in each overloaded operator function.
- Operator overloading is applicable within the scope (extent) in which overloading occurs.
- Only existing operators can be overloaded. We cannot create a new operator.
- C++ has wide range of operators. However, few operators cannot be overloaded to operate in the same manner like built-in operators. The operators given in Table 8.1 cannot be overloaded.

Table 8.1 Non-overloadable operators

Operator	Description
.	Member operator
.*	Pointer to member operator
::	Scope access operator

?:	Conditional operator
Sizeof()	Size of operator
# and ##	Preprocessor symbols

For example, operators such as ?:, :: and .* are combinations of more than one symbol. The condition operator needs three arguments. It is only one operator that requires three arguments. Hence, above operators cannot be overloaded.

- The operators given in Table 8.2 cannot be overloaded using friend function.
- When unary operators are overloaded using member functions, it requires no explicit friend argument and returns no value whereas when it is overloaded using friend function, it requires one reference argument.
- When binary operators are overloaded using friend functions, it requires two arguments whereas when overloaded using member function, it requires one argument

Operator	Description
()	Function call delimiter/operator
=	Assignment operator
[]	Subscripting operator
->	Class member access operator

one argument constructor and operator function

We know that a single argument constructor or an operator function could be used for conversion of objects of different classes. A large range of classes as class libraries are available with the compiler, however, they are linked with the main program. Their source code is invisible to us. Only objects of these classes can be used. The user cannot change the in-built classes. The problem occurs when programmer attempts conversion from object of class declared by him/her to type of in-built class. This problem can be avoided by defining conversion routine in the user-defined class. The conversion routine may be single argument constructor or an operator function. It depends on the object whether it is source or destination object. [Table 8.3](#) describes conversion type and place of routine to be defined, followed by description.

Table 8.3 Conversion types

Sr.No.	Conversion type	Routine in destination class	Routine in source class
A	Class to class	Constructor	Conversion function
B	Class to basic	Not applicable	Conversion function
C	Basic to class	Constructor	Not applicable

(A) In case both the source and destination objects are of user-defined type, the conversion routine can be carried out using operator function in source class or using constructor in destination class.

(B) If the user-defined object is a destination object, the conversion routine should be carried out using single argument constructor in the destination object's class.

(C) In case the user-defined object is a source object, the conversion routine should be carried out using an operator function in the source object's class.

Defining multiple conversion routines puts the compiler in an uncertain condition. The compiler fails to select appropriate conversion routines. For example, if one argument constructor is present in destination class and operator function in source class, the compiler cannot select appropriate routines. Hence, while defining conversion routines, follow the conditions given in [Table](#)

overloading stream operators.

The predefined objects cin and cout are used to perform various input/ output operations in C++. The extraction operator (>>) is used with cin object to carry out input operations. The insertion operator (<<) is used with cout object to carry out output operations. In Chapter ***“Input and Output in C++”***, we learnt how to create objects similar to cin and cout. It is also possible to overload both these extraction and insertion operator with friend function.

The syntax for overloading (<<) insertion operator is as follows:

```
friend ostream & operator << ( ostream & put , v1)
{
    // code
    return put;
}
```

The keyword friend precedes the declaration. The ostream is an output stream class followed by reference and keyword operator. The put is an output stream object like cout. The v1 is a user-defined class object. The following program explains overloading of insertion operator with friend function.

8.17 Write a program to overload insertion operator (<<) with friend function.

```
# include <iostream.h>
# include <ostream.h>

class string
{
    char *s;
public:

    string ( char *k)
    {
        s=k;
    }

    friend ostream & operator << (ostream &put, string & k) {
        put <<k.s;
        return put;
    }
};
```

```
int main()
{
    clrscr( );
    string s("INDIA");
    cout<<s;
    return 0;
}
```

OUTPUT**INDIA**

Explanation: In the above program, the insertion operator << is overloaded with friend function. The overloaded operator allows us to display contents of objects directly using cout statement. The statement cout<<s; displays contents of object s on the screen. In the same way, extraction operator can be overloaded. The syntax for overloading extraction operator follows below:

```
friend istream & operator >> ( istream & get, v2)
{
    // code
    return get;
}
```

The following program explains overloading of extraction operator.

8.18 Write a program to overload extraction operator using friend function.

```
# include <iostream.h>
# include <constream.h>

class string
{
    char *s;

    public:

    friend istream & operator >> (istream &get, string & k)
    {
        cout <<"\n Enter a string : ";
        get >>k.s;
        return get;
    }

};

int main( )
{
    clrscr( );
    string s;
    cin>>s; // input string
    return 0;
}
```

}

OUTPUT**BEST LUCK**

Explanation: This program is the same as the last one. Here, extraction operator is overloaded. The object `s` is directly used with `cin` statement. After execution of this statement, the overloaded operator is invoked.

Inheritance:

Introduction

Inheritance is one of the most useful and essential characteristics of object-oriented programming. The existing classes are main components of inheritance. The new classes are created from existing one. The properties of existing classes are simply extended to the new classes. The new classes created using such methods are known as derived classes and the existing classes are known as base classes as shown in [Figure 9.1](#). The relationship between the base and derived class is known as *kind of relationship*. The programmer can define new member variables and functions in the derived class. The base class remains unchanged. The object of derived class can access members of base as well as derived class. On the other hand, the object of base class cannot access members of derived classes. The base classes do not know about their subclasses.

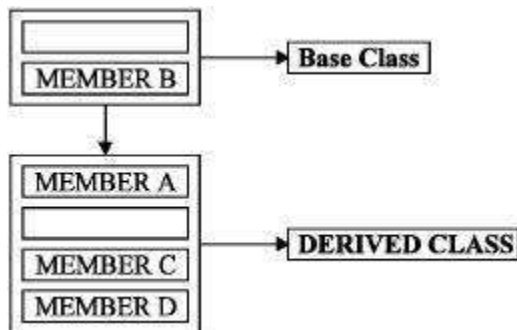


Fig. 9.1 Inheritances

The term **reusability** means reuse of properties of base class in the derived classes. Reusability permits to reuse members of the previous class. In [Figure 9.1](#) the base class is reused. Reusability is achieved using inheritance. Inheritance and reusability are not different from each other. The outcome of inheritance is reusability.

This chapter deals with implementation of the mechanism of inheritance and understand the features of it by illustrating several examples. The property of inheritance is slightly different in C++ as compared to other languages. C++ allows inheriting all the properties of previous class and there is a flexibility to add new members in the derived class, which are not available in the parent class.

The base class is called as superclass, parent or ancestor and derived class as subclass, child or descendent. It is also possible to derive a class from previously derived class. A class can be derived from more than one class.

INHERITANCE

The procedure of creating a new class from one or more existing classes is termed as inheritance.

Types of Inheritance

So far we've learnt examples of simple inheritance that uses one base class and one derived class. The process of inheritance can be a simple one or may be complex.

This depends on the following points:

(1) Number of base classes: The program can use one or more base classes to derive a single class.

(2) Nested derivation: The derived class can be used as base class and new class can be derived from it. This can be possible to any extent.

Depending on the above points inheritance is classified as follows:

- Single Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Multilevel Inheritance
- Hybrid Inheritance
- Multi-path Inheritance

The different types of inheritance are described in [Figure 9.6](#). The base classes are at the top level and derived classes at the bottom. The arrow pointed from top to bottom indicates that properties of base classes are inherited by the derived class and the reverse is not applicable.

(1) SINGLE INHERITANCE

When only one base class is used for derivation of a class and the derived class is not used as base class, such type of inheritance between one base and derived class is known as **single inheritance**. [Figure 9.6](#) (a) indicates single inheritance.

(2) MULTIPLE INHERITANCE

When two or more base classes are used for derivation of a class, it is called **multiple inheritance**. [Figure 9.6](#) (b) indicates multiple inheritance.

(3) HIERARCHICAL INHERITANCE

When a single base class is used for derivation of two or more classes, it is known as **hierarchical inheritance**. [Figure 9.6](#) (c) indicates hierarchical inheritance.

(4) MULTILEVEL INHERITANCE

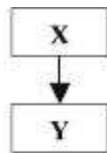
When a class is derived from another derived class i.e., derived class acts as a base class, such type of inheritance is known as **multilevel inheritance**. [Figure 9.6](#) (d) indicates multilevel inheritance.

(5) HYBRID INHERITANCE

The combination of one or more type of inheritance is known as **hybrid inheritance**. [Figure 9.6](#) (e) indicates hybrid inheritance.

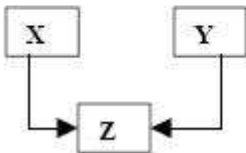
(6) MULTIPATH INHERITANCE

When a class is derived from two or more classes that are derived from same base class such type of inheritance is known as **multipath inheritance**. [Figure 9.6](#) (f) indicates multipath inheritance.



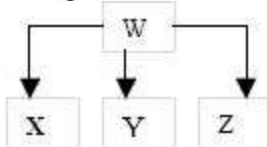
X is a base class. Y is a derived class. This type involves one base and derived class. Further, no class is derived from Y.

(a) Single Inheritance



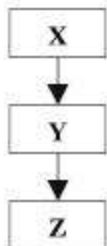
X and Y are base classes. Z is a derived class. Class Z inherits properties of both X and Y. Further, Z is not used as a base class.

(b) Multiple Inheritance



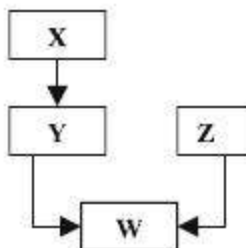
W is only one base class. X, Y and Z are derived classes. Further, X, Y and Z are not used for deriving a class.

(c) Hierarchical Inheritance



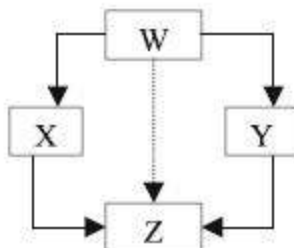
X is a base class. Y is derived from X. Further, Z is derived from Y. Here, Y is not only a derived class but also a base class. Further, Z can be used as a base class.

(d) Multilevel Inheritance



In this type, two types of inheritance is used i.e., single and multiple inheritance. Class Y is derived from class X. It is a single type of inheritance. Further, the derived class Y acts as a base class. The class W is derived from base classes Y and Z. This type of inheritance that uses more than one base class is known as multiple inheritance. Thus, combination of one or more type of inheritance is called as hybrid inheritance.

(e) Hybrid Inheritance



W is a base class. The classes X and Y are derived from W base class. Both X and Y inherit properties of class W. Further, class Z is derived from X and Y. X and Y have same copies of members inherited from W. Here, ambiguity is generated. Hence, virtual keyword is used to avoid ambiguity. The virtual mechanism is explained later.

(f) Multipath Inheritance

Fig. 9.6 Types of Inheritances

9.5 SINGLE INHERITANCE

When only one class is derived from a single base class such derivation of a class is known as **single inheritance**, further, the derived class is not used as a base class. This type of inheritance uses one base and one derived class.

The new class is termed as derived class and the old class is called as base class as shown in [Figure 9.7](#). A derived class inherits data member variables and functions of base class.

However, constructors and destructors of base class are not inherited in derived class.

The newly created class receives entire characteristics from its base class. In single inheritance, there are only one base class and derived class. The single inheritance is not as complicated as compared to other types of inheritances.

In the above diagram, class ABC is a base class and class abc is derived class. The arrow shows that class *abc* is derived from class ABC. The program given below illustrates single inheritance.

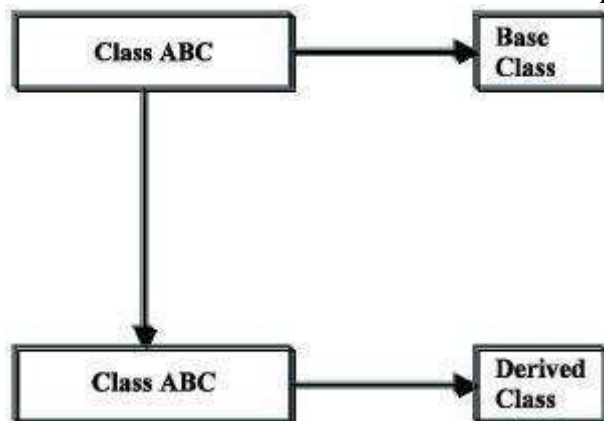


Fig. 9.7 Single inheritance

9.5 Write a program to show single inheritance between two classes.

```

#include <iostream.h>
#include <constream.h>
class ABC
{
    protected:
    char name[15];
    int age;
};

class abc : public ABC    // public derivation
{
    float height;
    float weight;

public:

    void getdata( )
    {

```

```

cout <<"\n Enter Name and Age : ";
cin >>name>>age;

cout <<"\n Enter Height and Weight : ";
cin >>height >>weight;
}

void show( )
{
    cout <<"\n Name   : "<<name <<"\n Age   : "<<age<<" Years";
    cout <<"\n Height : "<<height <<" Feets"<<"\n Weight : 
        " <<weight <<" Kg.";
}
};

void main( )
{
    clrscr( );
    abc x;
    x.getdata( );    // Reads data through keyboard.
    x.show( );       // Displays data on the screen.
}

```

OUTPUT**Enter Name and Age : Santosh 24****Enter Height and Weight : 4.5 50****Name : Santosh****Age : 24 Years****Height : 4.5 Feets****Weight : 50 Kg.**

Explanation: In the above program, two classes ABC and abc are declared. The class ABC has two protected data members name and age. The class abc has two float data members height and weight with two-member functions getdata () and show(). The class abc is derived from class ABC. The statement class abc: public ABC defines the derived class abc. In function main(), x is an object of derived class abc. The object x invokes member functions getdata() and show(). This function reads and displays data respectively.

96 MULTILEVEL INHERITANCE

The procedure of deriving a class from derived class is named as **multilevel inheritance**.

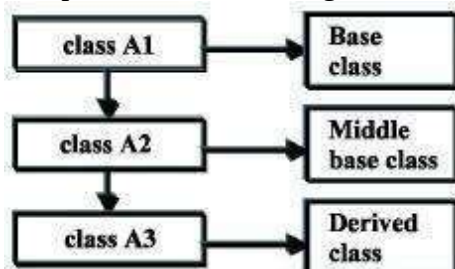


Fig. 9.8 Multilevel inheritance

In the [Fig. 9.8](#) class A3 is derived from class A2. The class A2 is derived from class A1. The class A3 is derived class. The class A2 is a derived class as well as base class for class A3. The class A2 is called as intermediate base class. The class A1 is a base class of classes A2 and A3. The series of classes A1, A2, and A3 is called as inheritance pathway as shown in [Figure 9.8](#).

9.6 Write a program to create multilevel inheritance. Create classes A1, A2, and A3.

```
// Multilevel inheritance //
```



```
#include <iostream.h>
#include <constream.h>

class A1          // Base class
{
    protected :
    char name[15];
    int age;
};

class A2 : public A1 // Derivation first level
{
    protected :
    float height;
    float weight;
};

class A3 : public A2  // Derivation second level
{
    protected :
    char sex;
    public :

    void get( )      // Reads data
    {
        cout <<"Name   : "; cin >>name;
        cout <<"Age    : "; cin >>age;
        cout <<"Sex    : "; cin >>sex;
        cout <<"Height : "; cin >>height;
        cout <<"Weight : "; cin >>weight;
    }

    void show( )    // Displays data
```

```
{
    cout <<"\nName : " <<name;
    cout <<"\nAge   : " <<age <<" Years";
    cout <<"\nSex   : " <<sex;
    cout <<"\nHeight : " <<height <<" Feets";
    cout <<"\nWeight : " <<weight <<" Kg.";
}
};

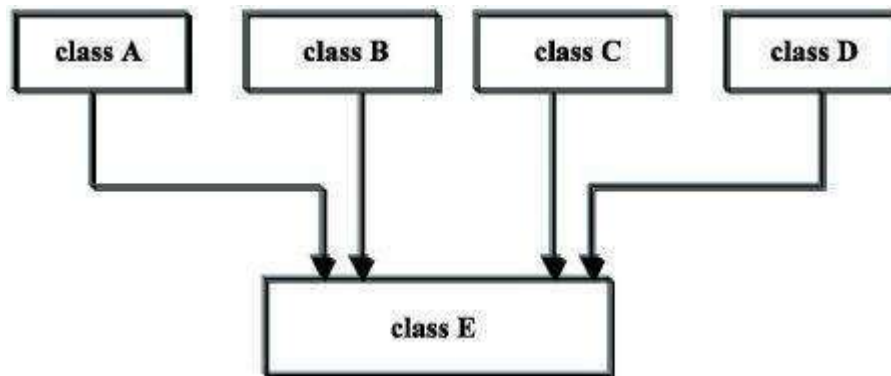
void main( )
{
    clrscr( );
    A3 x;      // Object Declaration
    x.get( );  // Reads data
    x.show( ); // Displays data
}
```

OUTPUT**Name : Balaji****Age : 26****Sex : M****Height : 4****Weight : 4.9****Name : Balaji****Age : 26 Years****Sex : M****Height : 4 Feets****Weight : 4.9 Kg.**

Explanation: In the above program, the class A1, A2, and A3 are declared. The member variables of all these classes are protected. The class A2 is derived from class A1. The class A3 is derived from class A2. Thus, the class A2 acts as derived class as well as base class. The function get() reads data through the keyboard and function show() displays data on the screen. Both the functions are invoked using object x of class A3.

9.7 MULTIPLE INHERITANCE

Multiple inheritance is the latest addition to the C++ language. When a class is derived from more than one class then this type of inheritance is called as **multiple inheritance**. A class can be derived by inheriting properties of more than one class. Properties of various pre-defined classes are transferred to single derived class. [Figure 9.9](#) shows multiple inheritance.

**Fig.9.9** Multiple inheritance

9.7 Write a program to derive a class from multiple base classes.

// Multiple Inheritance //

```
#include <iostream.h>
#include <constream.h>
```

```
class A { protected : int a; }; // class A declaration
class B { protected : int b; }; // class B declaration
class C { protected : int c; }; // class C declaration
class D { protected : int d; }; // class D declaration
```

```
// class E : public A, public B, public C, public D
```

```
class E : public A,B,C,D // Multiple derivation
{
    int e;
    public :
```

```
    void getdata( )
    {
        cout <<"\n Enter values of a,b,c & d & e : ";
        cin >>a>>b>>c>>d>>e;
    }
```

```
    void showdata( )
    {
```

```
        cout <<"\n a="<<a <<" b = "<<b <<" c = "<<c <<" d= "<<d <<" e= "<<e;
    }
```

```
};

void main ( )
{
    clrscr( );
    E x;
    x.getdata( ); // Reads data
    x.showdata( ); // Displays data
}
```

OUTPUT

Enter values of a,b,c & d & e : 1 2 4 8 16

a=1 b = 2 c = 4 d= 8 z= 16

Explanation: In the above program, classes A, B, C, D, and E are declared each with one integer member variable. The class E has two-member functions, getdata() and showdata().

The getdata() is used to read integers through the keyboard and showdata() is used to display the contents on the screen. The class E is derived from the classes A, B, C, and D. The classes A, B, C, and D all together act as a base class. The derivation is carried out with the statement class E: public A, B, C, and D. The class members of A are publicly derived and members of other classes are privately derived. To derive all class members publicly, the statement would be as class Z: public A, public B, public C, public D. Remember that the meaning of both the statements is not the same. Most programmers refer to the second format as it reduces the code.

98 HIERARCHICAL INHERITANCE

We know that in inheritance, one class could be inherited from one or more classes. In addition, new members are added to the derived class. Inheritance also supports hierarchical arrangement of programs. Several programs require hierarchical arrangement of classes, in which derived classes share the properties of base class. Hierarchical unit shows top down style through splitting a compound class into several simple sub classes. Figure 1.16 given in

Chapter “*Introduction to C++*”, is a perfect example of hierarchy of classes. The program based on hierarchical inheritance is illustrated below.

9.8 Write a program to show hierarchical inheritance.

```
# include <constream.h>
# include <iostream.h>
```

```
class red
{
    public:
    red ( ) {cout<<" Red ";};
};
```

```
class yellow
{
    public :
```

```
yellow( ) { cout <<" Yellow "; }  
};  
  
class blue  
{  
    public:  
    blue ( ) { cout <<" Blue "; }  
};  
  
class orange : public red, public yellow  
{  
    public :  
    orange( ) {    cout <<" = Orange ";    }  
};  
  
class green : public blue, public yellow  
{  
    public:  
    green( ) { cout <<" = Green "; }  
  
};  
  
class violet : public red, public blue  
{  
    public:  
    violet( ) { cout <<" = Violet "; }  
};  
  
class reddishbrown : public orange, public violet  
{  
    public:  
    reddishbrown( ) { cout <<" = Reddishbrown "; }  
};  
  
class yellowishbrown : public green, public orange  
{  
    public:  
    yellowishbrown( ) { cout<<" = Yellowishbrown "; }  
};
```

```

class bluishbrown : public violet, public green
{
    public:
    bluishbrown( ) { cout<<" = Bluishbrown "; }
};

void main( )
{
    clrscr( );
    reddishbrown r;
    endl(cout);

    bluishbrown b;
    endl(cout);

    yellowishbrown y;
    endl(cout);
}

```

OUTPUT

Red Yellow = Orange Red Blue = Violet = Reddishbrown

Red Blue = Violet Blue Yellow = Green = Bluishbrown

Blue Yellow = Green Red Yellow = Orange = Yellowishbrown

Explanation: For each colour separate class with constructor is declared. The classes red, blue, and yellow are base classes. The class orange is derived from classes red and yellow. The class green is derived from blue and yellow. The class violet is derived from red and blue. The class reddishbrown is derived from classes orange and violet. The class yellowishbrown is derived from classes green and orange and lastly the class bluishbrown is derived from violet and green.

In function main () , objects of classes reddishbrown , yellowishbrown, and bluishbrown are declared. When objects are declared constructors are executed from base to derived classes. Constructor when executed displays the class name (colour name). The output shows names of different colours and resulting colour after their combination.

99 HYBRID INHERITANCE

The combination of one or more types of inheritance is known as **hybrid inheritance**. Sometimes, it is essential to derive a class using more types of inheritances. [Figure 9.10](#) shows hybrid inheritance. In the diagram given below the class GAME is derived from two base classes i.e., LOCATION and PHYSIQUE. The class PHYSIQUE is also derived from class player.

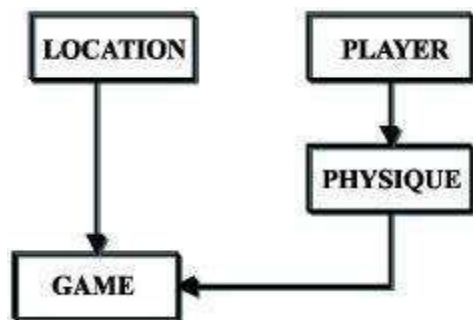


Fig. 9.10 Hybrid inheritance

9.9 Write a program to create a derived class from multiple base classes.

// Hybrid Inheritance //

```
# include <iostream.h>
# include <constream.h>
```

```
class PLAYER
{
protected :
char name[15];
char gender;
int age;
};
```

```
class PHYSIQUE : public PLAYER
{
protected :
float height;
float weight;
};
```

```
class LOCATION
{
protected :
char city[10];
char pin[7];
};
```

```
class GAME : public PHYSIQUE, LOCATION
{
protected :
char game[15];
public :
```

```
void getdata( )
{
    cout <<" Enter Following Information\n";
    cout <<"Name   : "; cin>> name;
    cout <<"Gender : "; cin>>gender;
    cout <<"Age    : "; cin>>age;
    cout <<"Height : "; cin>>height;
    cout <<"Weight : "; cin>>weight;
    cout <<"City   : "; cin>>city;
    cout <<"Pincode : "; cin>>pin;
    cout <<"Game    : "; cin>>game;
}
void show( )
{
    cout <<"\n Entered Information";
    cout <<"\nName   : "; cout<<name;
    cout <<"\nGender : "; cout<<gender;
    cout <<"\nAge    : "; cout<<age;
    cout <<"\nHeight : "; cout<<height;
    cout <<"\nWeight : "; cout<<weight;
    cout <<"\nCity   : "; cout<<city;
    cout <<"\nPincode : "; cout<<pin;
    cout <<"\nGame    : "; cout<<game;
}
};
```

```
int main( )
```

```
{
    clrscr( );
    GAME G;
    G.getdata( );
    G.show( );
    return 0;
}
```

OUTPUT

Enter Following Information

Name : Mahesh

Gender : M

Age 25

Height : 4.9

Weight 55

City : Nanded

Pincode : 431603

Game : Cricket

Entered Information**Name : Mahesh****Gender : M****Age : 25****Height : 4.9****Weight : 55****City : Nanded****Pincode : 431603****Game : Cricket**

Explanation: In the above program, PLAYER, PHYSIQUE, LOCATION, and GAME classes are defined. All the data members of these four classes are protected. The class PHYSIQUE is derived from base class PLAYER. The class GAME is derived from PHYSIQUE and LOCATION i.e., the derived class GAME has two base classes PHYSIQUE and LOCATION. The class LOCATION is a separate class and not derived from any other class. The getdata() and show() are functions of class GAME which read and display data respectively. In main(), the object G of class GAME calls these functions one by one to read and write the data. The output of the program is shown at the end of the above program.

9.10 MULTIPATH INHERITANCE

When a class is derived from two or more classes, which are derived from the same base class such type of inheritance is known as **multipath inheritance**. Multipath inheritance consists of many types of inheritances such as multiple, multilevel and hierarchical as shown in [Figure 9.11](#).

Consider the following example:

```
class A1
{
    protected:
    int a1;
};

class A2 : public A1
{
    protected:
    int a2;
};

class A3: public A1
{
    protected: int a3;
};

class A4: public A2,A3
{int a4;      };
```

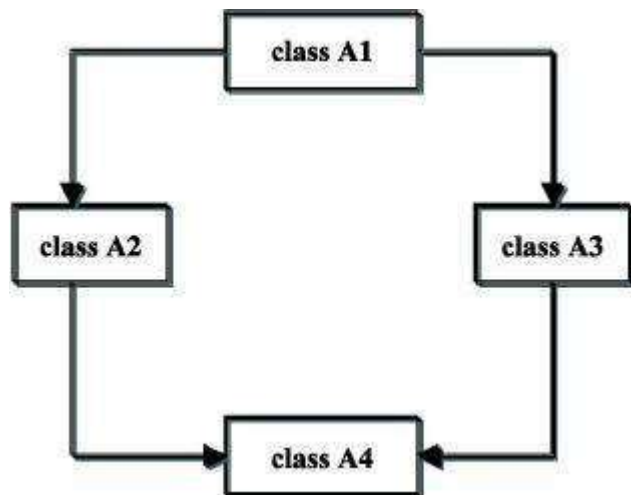


Fig. 9.11 Ambiguity in classes

In the given example, class A2 and A3 are derived from the same base class i.e., class A1 (hierarchical inheritance). The classes A2 and A3 both can access variable a1 of class A1. The class A4 is derived from class A2 and class A3 by multiple inheritances. If we try to access the variable a1 of class A, the compiler shows error messages as given below.

(a) Error PRG58.CPP 30: Member is ambiguous: 'A1::a1' and 'A1::a1'

(b) Error PRG58.CPP 37: Member is ambiguous: 'A1::a1' and 'A1::a1'

In the above example, we can observe all types of inheritances i.e. multiple, multilevel and hierarchical. The derived class A4 has two sets of data members of class A1 through the middle base classes A2 and A3. The class A1 is inherited twice.

Virtual base classes

To overcome the ambiguity occurred due to multipath inheritance, C++ provides the keyword `virtual`. The keyword `virtual` declares the specified classes virtual. The example given below illustrates the virtual classes.

```

class A1
{
    protected:
    int a1;
};

class A2 : virtual public A1    // virtual class declaration
{
    protected:
    int a2;
};

class A3: virtual public A1    // virtual class declaration
{
    protected:
    int a3;
};
  
```

```
class A4: public A2,A3
{
int a4;
};
```

When classes are declared as virtual, the compiler takes necessary precaution to avoid duplication of member variables. Thus, we make a class virtual if it is a base class that has been used by more than one derived class as their base class.

Let us consider an example that will give a clear idea of virtual classes as well as necessary care to be taken while declaring them.

(a) A base class cannot be specified more than once in a derived class.

```
class A
{ _____
_____};
```

```
class A1: A, A { ... }; // Illegal Declaration
```

Here, class A is specified twice. Hence, it is an illegal declaration.

(b) A base class can be indirectly passed to the derived class more than once.

```
class L: public A { ... }
class K: public A { ... }
class J: public L, public K { ... } // Legal declaration
```

In the above case (b), each object of class J will contain two sub-objects of class A through classes L and K.

(c) Case (b) causes some problems. To avoid duplication we can add the keyword virtual to a base class specifier.

For example,

```
class L : virtual public A { ... }
class K : virtual public A { ... }
class J : public K, public J { ... }
```

The virtual keyword always appears before the class name. A is now a virtual base class, and class J has only one sub-object of class A.

9.10 Write a program to declare virtual base classes. Derive a class using two virtual classes.

```
// VIRTUAL BASE CLASSES //
```

```
# include <iostream.h>
# include <conio.h>
```

```
class A1
{
protected:
int a1;
};
```

```
class A2 : public virtual A1 // virtual declaration
{
```

```

protected :
int a2;
};

class A3 : public virtual A1 // virtual declaration
{
protected:
int a3;
};

class A4 : public A2,A3 // virtual declaration
{
int a4;
public :

void get( )
{
cout <<"Enter values for a1, a2,a3 and a4 : ";
cin >>a1>>a2>>a3>>a4;
}

void put( )
{
cout <<"a1= "<<a1 <<" a2 = "<<a2 <<" a3 = "<<a3 <<"a4 = "<<a4;
}
};

void main( )
{

clrscr( );
A4 a;
a.get( ); // Reads data
a.put( ); // Displays data
}

```

OUTPUT**Enter values for a1 a2,a3 and a4 : 5 8 7 3****a1= 5 a2 = 8 a3 = 7 a4 = 3**

Explanation: In the above program, the classes A1, A2, A3, and A4 are declared and each contains one protected member variable. The class A4 has two member functions get() and put(). The get() function reads integers through the keyboard. The put() function displays the contents of the member variables on the screen. The classes A2 and A3 are derived from class A1. While deriving these two classes, the class A1 is declared as virtual as per the following statements:

- (a) class A2 : public virtual A1 // derivation of class A2
- (b) class A3 : public virtual A1 // derivation of class A3

(c) class A4: public A2,A3 // derivation of class A4

The class A4 is derived from two classes A2 and A3 as per the statement (c). In function main(), the object a of class A4 invokes the member function get() and put().

constructors and destructors and inheritance

The constructors are used to initialize member variables of the object and the destructor is used to destroy the object. The compiler automatically invokes constructors and destructors. The derived class does not require a constructor if the base class contains zero-argument constructor.

In case the base class has parameterized constructor then it is essential for the derived class to have a constructor. The derived class constructor passes arguments to the base class constructor.

In inheritance, normally derived classes are used to declare objects. Hence, it is necessary to define constructor in the derived class. When an object of a derived class is declared, the constructors of base and derived classes are executed.

In inheritance, destructors are executed in reverse order of constructor execution. The destructors are executed when an object goes out of scope. To know the execution of constructor and destructor let us study the following program.

9.11 Write a program to show sequence of execution of constructor and destructor in multiple inheritance.

```
# include <iostream.h>
# include <constream.h>
```

```
class A // base class
```

```
{
public :
A ( )
{ cout << "\n Zero-argument constructor of base class A"; }
```

```
~A( )
```

```
{ cout << "\n Destructor of the class A"; }
```

```
};
```

```
class B // base class
```

```
{
public:
B( )
{ cout << "\n Zero-argument constructor of base class B "; }
```

```
~B( ) { cout <<"\n Destructor of the class B " ; }
};

class C : public A, public B    // Derivation of class
{
    public:
    C( )
    { cout <<"\n Zero-argument constructor of derived class C";
    }

    ~C( ) { cout<<"\n Destructor of the class C"; }

};

void main( )
{
    clrscr( );
    C objc; // Object declaration
}
```

OUTPUT

Zero-argument constructor of base class A
Zero-argument constructor of base class B
Zero-argument constructor of derived class C
Destructor of the class C
Destructor of the class B
Destructor of the class A

Explanation: In the above program, class A and B are two classes. The class C is derived from classes A and B. The constructors of base classes are executed first followed by derived class. The destructors of derived classes are executed first followed by base class.

(1) BASE AND DERIVED CLASSES WITH CONSTRUCTOR

9.12 Write a program to declare both base and derived classes with constructors.

```
# include <iostream.h>
# include <conio.h>
```

```
class I
{
    public :
```

```
I( )
```

```

{
    cout << "\n In base class constructor "; }
};

class II : public I
{
    public:
    II() { cout << "\n In derived class constructor\n";
}

};

void main( )
{
    clrscr( );
    II i;
}

```

OUTPUT

In base class constructor

In derived class constructor

Explanation: In the above program, both the base and derived classes contain constructors.

When the object of derived class type is declared and constructors of both the classes are executed, the constructor of base class is executed first and then the constructor of derived class.

(2) BASE CLASS WITH VARIOUS CONSTRUCTORS AND DERIVED CLASS WITH ONE CONSTRUCTOR

913 Write a program to declare multiple constructors in base class and single constructor in derived class.

```

# include <iostream.h>
# include <conio.h>

```

```

class I
{
    public :
    int x;

    I() { cout << "\nZero argument base class constructor "; }

    I( int k)
    {

```

```

    cout <<"\nOne argument base class constructor";
}

};

class II : public I
{
    int y;
public:

    II (int j)
    {
        cout <<"\nOne argument derived class constructor ";
        y=j;
    }
};

void main( )
{
    clrscr( );
    II i(2);
}

```

OUTPUT

Zero argument base class constructor

One argument derived class constructor

Explanation: In the above program, the class I has zero and one argument constructor. The class II has only one argument constructor. The object i is declared with an integer. The zero argument constructor of base class and one argument constructor of derived class are executed.

(3) BASE AND DERIVED CLASSES WITHOUT DEFAULT CONSTRUCTOR

914 Write a program to declare base and derived class without default constructor.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```

class I
{
    public :
    int x;

```

```

I ( int k)
{
x=k;
cout <<"\nOne argument base class constructor"; }

};

class II : public I
{
int y;
public:

II (int j) : I(j)
{
cout <<"\nOne argument derived class constructor ";

y=j;
}
};

void main( )
{
clrscr( );
II i(2);
}

```

OUTPUT

One argument base class constructor
One argument derived class constructor

Explanation: In the above program, no default constructor is declared. In class II one argument constructor is declared and base class constructor is explicitly invoked. When object i is declared, one argument constructor of both the classes is executed. In the absence of explicit call of base class constructor the compiler will display the error message “Cannot find default constructor to initialize base class ‘I’”.

(4) CONSTRUCTORS AND MULTIPLE INHERITANCE

915 Write a program to derive a class using multiple base classes. Observe the execution of constructor when object of derived class is declared.

```

# include <iostream.h>
# include <conio.h>

```

```

class I
{

```

```
public:

I( ) { cout <<"\n Zero argument constructor of base class I "; }

};

class II
{
public:

    II( ) { cout<<"\n Zero argument constructor of base class II "};

};

class III : public II,I
{
public:

    III( )
    { cout<<"\n Zero argument constructor of base class III "; }

};

void main( )
{
    clrscr( );
    III i;
}
```

OUTPUT

```
Zero argument constructor of base class II
Zero argument constructor of base class I
Zero argument constructor of base class III
```

Explanation: The classes I and II are base classes of derived class III. In function main(), i is an object of derived class III. The execution of constructors depends on the sequence given while deriving a class as per the following statement:

class III : public II,I

Here, the class II is the first base class and class I is the second base class and the execution sequence of constructors is shown in the output.

(5) CONSTRUCTORS IN MULTIPLE INHERITANCES WITH EXPLICIT CALL

9.16 Write a program to derive a class using multiple base classes. Invoke the constructors of base classes explicitly.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```
class I
{
    public:
```

```
I( ) { cout <<"\n Zero argument constructor of base class I "; }
```

```
};
```

```
class II
{
    public:
```

```
II( ) { cout<<"\n Zero argument constructor of base class II ";}
};
```

```
class III : public II,I
{
    public:
```

```
III( ) : II( ), I( )
{ cout<<"\n Zero argument constructor of base class III "; }
};
```

```
void main( )
{
    clrscr( );
    III i;
}
```

OUTPUT

Zero argument constructor of base class II

Zero argument constructor of base class I

Zero argument constructor of base class III

Explanation: In this program, in class III explicitly, the constructors of both the base classes II and I are invoked. The execution sequence can be observed in the output. The execution sequence of constructor depends on the sequence of base classes and not according to the explicit calls.

(6) MULTIPLE INHERITANCE AND VIRTUAL CLASS

9.17 Write a program to derive a class using multiple base classes. Invoke the constructors of base classes explicitly. Declare any one base class as virtual.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```
class I
```

```
{
```

```
public:
```

```
I() { cout << "\n Zero argument constructor of base class I "; }
```

```
};
```

```
class II
```

```
{
```

```
public:
```

```
II() { cout << "\n Zero argument constructor of base class II "; }
```

```
};
```

```
class III : public II, virtual I
```

```
{
```

```
public:
```

```
III() : II(), I()
```

```
{ cout << "\n Zero argument constructor of base class III "; }
```

```
};
```

```
void main( )
```

```
{
```

```
clrscr( );
```

```
III i;
```

```
}
```

OUTPUT

Zero argument constructor of base class I

Zero argument constructor of base class II

Zero argument constructor of base class III

Explanation: In this program, the base class I is declared as virtual class while deriving the class. The constructor of virtual class is executed first. The execution of constructor here is not according to the sequence of base class.

(7) EXECUTION OF CONSTRUCTORS IN MULTILEVEL INHERITANCE

9.18 Write a program to derive a class using multilevel inheritance and observe the execution sequence of constructors.

include <iostream.h>

include <conio.h>

class I

{

public:

I() { cout <<"\n Zero argument constructor of base class I "; }

};

class II : public I

{

public:

II() { cout<<"\n Zero argument constructor of base class II ";}

};

class III : public II

{

public:

III() { cout<<"\n Zero argument constructor of base class III "; }

};

void main()

{

clrscr();

III ii;

}

OUTPUT

Zero argument constructor of base class I

Zero argument constructor of base class II

Zero argument constructor of base class III

Explanation: In this program, the class II is derived from class I. The class III is derived from class II. The class II is base as well as derived class. In function main (), ii is an object of

classIII. The constructors are executed from base to derived classes as shown in the output. Table 9.3 shows order of execution of constructors.

Table 9.3 Execution sequences of constructors

Statements	Sequence of execution	Remarks
Class II : public I	I () – Base class constructor II ()- Derived class constructor	Single inheritance
Class III: public I, II	I () – Base class constructor II ()- Base class constructor III ()- Derived class constructor	Multiple inheritance
Class III: public I, virtual II	II () – Virtual class constructor I () – Base class constructor III () – Derived class constructor	Multiple inheritance
class II : public I { };	I () – First base class constructor II () – second base class constructor III () – derived class constructor	Multiple inheritance
class III: public II { };		

9.13 OBJECT AS A CLASS MEMBER

Properties of one class can be used in another class using inheritance or using object of a class as a member in another class. Declaring object as a class data member in another class is also known as delegation. When a class has an object of another class as its member, such class is known as container class.

In inheritance, derived classes can use members of base class. Here, the derived class is a kind of base class. The programmer can also add new members to the derived class.

In delegation, the class is composed from objects of another classes. The composed class uses properties of other classes through their objects. This kind of relationship is known as has-a-relationship or containership.

class I

```
{
    ***** // members
    *****
}
```

class II

```
{
    I J; // object of class I as member
    *****
}
```

9.19 Write a program to use object of one class in another class as a member.

```
# include <iostream.h>
# include <conio.h>
```

```
class I
{
```

```
public:
int x;

I()
{
    cout <<"\n Constructor of class I";
    x=20;
}
};

class II
{
public:
int k;
I y;

II()
{
    k=30;
    cout <<"\n Constructor of class II";
}

void show( ) { cout <<"\n x= "<<y.x <<" k = "<<k; }
};

void main( )
{
    clrscr( );
    II ii;
    ii.show( );
}
OUTPUT
Constructor of class I
Constructor of class II
x= 20 k = 30
```

Explanation: In the above program, class II contains integer k and object y of class I. In function main(), object ii is an object of class II. The constructor of class I is executed first because when compiler reaches class II, it finds an object of class I. We know that object declaration always executes constructors of that class. Thus, object of class I is declared and constructor of class I is executed. The constructor of class II is executed and variable of

class II is initialized. The member function show() displays the contents of x and k. The contents of class I member is obtained from object y. The dot (.) operator is used to access the elements of class I.

9.20 Write a program to access member variables of base class using object, scope access operator and direct.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```
class A1
```

```
{
```

```
    public :
```

```
    char name[15];
```

```
    int age;
```

```
};
```

```
class A2 : public A1
```

```
{
```

```
    private
```

```
    : A1 a;
```

```
    float height;
```

```
    float weight;
```

```
    public :
```

```
    A2( )
```

```
    {
```

```
        clrscr( );
```

```
        cout <<"Access Using Scope Access operator\n";
```

```
        cout <<"Name   : "; cin>>A1::name;
```

```
        cout <<"Age    : "; cin>>A1::age;
```

```
        cout <<"Access Using object of the class\n";
```

```
        cout <<"Name : "; cin>>a.name;
```

```
        cout <<"Age   : "; cin>>a.age;
```

```
        cout <<"Access Using direct member variables\n";
```

```
cout <<"Name : "; cin >>name;
cout <<"Age    : "; cin >>age;

cout <<"Height : "; cin >>height;
cout <<"Weight : "; cin >>weight;
}

~A2()

{
    cout <<"\nDisplay using Scope Access operator\n";

    cout <<"\nName : " <<A1::name;
    cout <<"\nAge   : " <<A1::age;

    cout <<"\nDisplay Using object of the class\n";

    cout <<"\nName : " <<a.name;
    cout <<"\nAge   : " <<a.age;

    cout <<"\nAccess Using direct member variables\n";

    cout <<"\nName : " <<name;
    cout <<"\nAge   : " <<age;

    cout <<"\nHeight : " <<height;
    cout <<"\nWeight : " <<weight;
}
};

void main( )
{
    A2 x;
}
OUTPUT
Name : Ajay
```

```

Age : 21
Access Using object of the class
Name : Amit
Age : 20
Access Using direct member variables
Name : Arun
Age : 19
Height : 5.5
Weight : 31
Display using Scope Access operator
Name : Arun
Age : 19
Display Using object of the class
Name : Amit
Age : 20
Access Using direct member variables
Name : Arun
Age : 19
Height : 5.5
Weight : 31

```

Explanation: In the above program, A1 and A2 are two classes. The class A2 is derived from class A1. Class A1 has two public member variables. Class A2 has three private members. One of its member is an object of class A1 i.e., object a. The object a holds its separate set of member variables of class A1. There are three ways to access member variables of base class A1. Table 9.4 describes status of access of member variables of class A1 through derived class A2 when we use access specifiers such as private, public and protected.

Table 9.4 Access specifiers

Access (using)	Public	Private	Protected
Scope Access :: operator	Possible	Not Possible	Possible
Object	Possible	Not Possible	Not Possible
Direct	Possible	Not Possible	Possible

(1) The first way is to use scope access operator as given below:

```

cin>>A1::name;
cin>>A1::age;

```

In the above statements, A1 is a class name and name and age are member variables of class A1. When access specifier is public or protected the above statements are valid.

(2) The second method is to use objects of the base class as given below:

```

cin >>a.name;
cin >>a.age;

```

In the above statements, member variables of class A1 are accessed using objects of the same class, which is the member of the derived class. This is possible only when the member variables of class A1 are public and not possible if the member variables of class A1 are protected or private.

(3) The third method uses directly the member variables. Direct access is possible when access specifier is public or protected, as per the following statements.

```

cin >>name;

```

```
cin >>age;
```

9.21 Write a program to derive a class from two base classes. Use objects of both the classes as member variables for derived class. Initialize and display the contents of classes using constructor and destructor.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```
class A
{
    public :
    int a1;
};
```

```
class B
{
    public:
    int b1;
};
```

```
class AB
{
    public :
    A a;
    B b;
    public :
```

```
AB ( )
{
    a.a1=65;
    b.b1=66;
    cout <<"a1 = "<<a.a1 <<" b1 = "<<b.b1;
}
```

```
~AB( ){ };
};
```

```
int main( )
{
    clrscr( );
```

```
AB ab;
```

```
return 0;
}
```

```
OUTPUT
```

```
a1=65 b1=66
```

Explanation: In the above program, class A and class B are declared each with one integer. Class AB is declared which contains objects of A and B classes as member variables. The constructor of class AB initializes member variables of class A and B. The constructor also displays contents on the screen. Finally, the destructor destroys the object.

abstract classes

When a class is not used for creating objects it is called as abstract class. The abstract class can act as a base class only. It is a lay out abstraction in a program and it allows a base on which several levels of inheritance can be created. The base classes act as foundation of class hierarchy. An abstract class is developed only to act as a base class and to inherit and no objects of these classes are declared. An abstract class gives a skeleton or structure, using which other classes are shaped. The abstract class is central and generally present at the starting of the hierarchy. The hierarchy of classes means chain or groups of classes being involved with one another. In the last program, class A is an abstract class because no instance (object) of class A is declared.

qualifier classes and inheritance

The following program explains the behaviour of qualifier classes and the classes declared within them with inheritance.

9.22 Write a program to create derived class from the qualifier class.

```
# include <iostream.h>
```

```
# include <conio.h>
```

```
class A
```

```
{
```

```
    public:
```

```
    int x;
```

```
    A ( ) {}
```

```
        class B
```

```
        {
```

```
            public:
```

```
            int y;
```

```
            B ( ) {}
```

```
        };
```

```
}; class C : public A,A::B
```

```
{
```

```

    public:
    int z;

    void show( )
    {
        cout <<endl<<"x = "<<x <<" y ="<<y<<" z = "<<z;
    }

    C (int j,int k, int l)
    {
        x=j;
        y=k;
        z=l;
    }
};

void main( )
{
    clrscr( );
    C c(4,7,1);
    c.show( );
}

```

OUTPUT

x = 4 y =7 z = 1

Explanation: In the above program, class B is defined inside class A. The class A is a qualifier class of class B. The class C is inherited from the classes A and B. In the statement class C: public A, A::B, class C is inherited from A and B. To access class B, it is preceded by the qualifier class A and scope access operator. If we mention only class A, class B won't be considered for inheritance. Similarly, if we mention only class B, the qualifier class A will not be considered for inheritance.

COMMON CONSTRUCTOR

When a class is declared, constructor is also declared inside the class in order to initialize data members. It is not possible to use a single constructor for more classes. Every class has its own constructor and destructor with the same name as class. When a class is derived from another then it is possible to define a constructor in derived class and data members of both base and derived classes can be initialized. It is not essential to declare constructor in a base class. Thus, the constructor of the derived class works for its base class and such constructors are called as **common constructors**.

23 Write a program to initialize member variables of both base and derived classes using a constructor of derived class.

```

# include <iostream.h>
# include <conio.h>

```

```

class A
{
    protected:
    int x;
    int y;

};

class B : private A
{
    public:
    int z;

    B( ) { x=1,y=2,z=3;
        cout <<"x= "<<x << " y ="<<y <<" z="<<z; }

}
};

void main( )
{
    clrscr( );
    B b;
}

```

OUTPUT

x= 1 y =2 z=3

Explanation: In the above program, class A and class B are declared. Class B is derived from class A. The constructor of class B initializes member variables of both classes. Hence, it acts as a common constructor of both base and derived class.

common constructor

When a class is declared, constructor is also declared inside the class in order to initialize data members. It is not possible to use a single constructor for more classes. Every class has its own constructor and destructor with the same name as class. When a class is derived from another then it is possible to define a constructor in derived class and data members of both base and derived classes can be initialized. It is not essential to declare constructor in a base class. Thus, the constructor of the derived class works for its base class and such constructors are called as **common constructors**.

23 Write a program to initialize member variables of both base and derived classes using a constructor of derived class.

```
# include <iostream.h>
```

```
# include <conio.h>

class A
{
    protected:
    int x;
    int y;

};

class B : private A
{
    public:
    int z;

    B( ) { x=1,y=2,z=3;
        cout <<"x= "<<x <<" y ="<<y <<" z="<<z; }

}
};

void main( )
{
    clrscr( );
    B b;
}
OUTPUT
x= 1 y =2 z=3
```

Explanation: In the above program, class A and class B are declared. Class B is derived from class A. The constructor of class B initializes member variables of both classes. Hence, it acts as a common constructor of both base and derived class.

pointers and inheritance

The private and public member variables of a class are stored in successive memory locations. A pointer to public member variable gives us access to private member variables.

The same is true for derived class. The member variables of base class and derived class are also stored in successive memory locations. The following program explains the mechanism of accessing private data members of the base class using the address of public member variable of derived class using pointer. Here, no member functions are used.

24 Write a program to access private member variables of base class using pointers.

```
# include <iostream.h>
```

```
# include <conio.h>

class A
{
    private:
        int x;
        int y;

    public:

        A() {
            x=1;
            y=2;
        }
};
class B : private A
{
    public:
        int z;

        B() { z=3; }

};

void main( )
{
    clrscr( );
    B b; // object declaration
    int *p;// pointer declaration
    p=&b.z; // address of public member variabe is stored in pointer

    cout<<endl<<" Address of z : "<<(unsigned)p <<" "<<"Value of z :"<<*p;
    p--; // points to previous location
    cout<<endl<<" Address of y : "<<(unsigned)p <<" "<<"Value of y :"<<*p;
    p--;
    cout<<endl<<" Address of x : "<<(unsigned)p <<" "<<"Value of x :"<<*p;
}

OUTPUT
Address of z : 65524 Value of z :3
```

Address of y : 65522 Value of y :2

Address of x : 65520 Value of x :1

Explanation: In the above program, class A contains two private member variables x and y. The constructor initializes the member variables. Class B is derived from class A. The class B has one public member variable. In function main(), b is an object of class B. The pointer p is an integer pointer. The address of member variable z of derived class B is assigned to pointer p. By applying decrease operation, we get the previous memory locations where member variables of base classes are stored. The values of all member variables with their addresses are displayed. overloading member function.

The derived class can have the same function name as base class member function. An object of the derived class invokes member functions of the derived class even if the same function is present in the base class.

25 Write a program to overload member function in base and derived class.

```
# include <iostream.h>
```

```
# include <constream.h>
```

```
class B
{
    public:
```

```
    void show( )
    {
        cout <<"\n In base class function ";
    }
};
```

```
class D : public B
{
```

```
    public:

    void show( )
    {
        cout <<"\n In derived class function";
    }
};
```

```
int main( )
```

```
{  
  clrscr( );  
  B b;      // b is object of base class  
  D d;      // d is object of derived class  
  
  b.show( ); // Invokes Base class function  
  d.show( ); // Invokes Derived class function  
  d.B::show( ); // Invokes Base class function  
  return 0;  
}
```

OUTPUT

In base class function

In derived class function

In base class function

Explanation: In this program, class D is derived from class B. Both the classes have the same function `show( )` as member function. In function `main( )`, objects of both B and D classes are declared. The object `b` invokes the member function `show( )`. The object `b` is object of base class, hence, it invokes the member function `show( )` of base class. In addition, the objects of base class cannot invoke the member functions of derived class, because the base class does not have information about the classes derived under it.

The object `d` of derived class invokes the function `show( )`. When function with same name and argument list are present in both the base and derived classes, the objects of derived class give first priority to function of its own class. Thus, the statement `d.show( )` invokes function of derived class. To invoke the function of base class with the object of derived class, the class name and scope access operator are preceded before the matching function name. From the above program, it is also clear that we can declare objects of both base and derived classes. The objects of both base and derived classes are independent of each other.

ADVANTAGES OF INHERITANCE

- (1) The most frequent use of inheritance is for deriving classes using existing classes, which provides reusability. The existing classes remain unchanged. By reusability, the development time of software is reduced.
- (2) The derived classes extend the properties of base classes to generate more dominant object.
- (3) The same base classes can be used by a number of derived classes in class hierarchy.
- (4) When a class is derived from more than one class, all the derived classes have the same properties as that of base classes.

DISADVANTAGES OF INHERITANCE

- (1) Though object-oriented programming is frequently propagandized as an answer for complicated projects, inappropriate use of inheritance makes a program more complicated.
- (2) Invoking member functions using objects create more compiler overheads.
- (3) In class hierarchy various data elements remain unused, the memory allocated to them is not utilized.